porting_opencode

OpenCode → HelixCode Complete Porting Plan

Target: `github.com/HelixDevelopment/HelixCode` (`dev.helix.code`)
Source: `opencode-ai/opencode` (TypeScript/Go, 12K stars, 75+ providers)
Date: Auto-generated by porting specialist
Version: 1.0.0

Table of Contents

Feature 1: 75+ Provider Support with Mid-Session Switching

Source Location (in original agent)

- `opencode/src/llm/provider-registry.ts` — Model registration and discovery
- `opencode/src/llm/provider-switch.ts` — Mid-session model switching
- `opencode/src/llm/health-check.ts` — Provider health monitoring
- `opencode/src/llm/models.dev.ts` — models.dev provider catalog integration
- `opencode/src/llm/ai-sdk-wrapper.ts` — Vercel AI SDK wrapper

Target Location (in HelixCode)

- `internal/llm/provider_router.go` (NEW)
- `internal/llm/provider_registry.go` (NEW)
- `internal/llm/model_descriptor.go` (NEW)
- `internal/llm/provider_switcher.go` (NEW)
- `internal/llm/health_monitor.go` (NEW)
- `cmd/cli/main.go` (MODIFY)
- `internal/llm/llm.go` (MODIFY)
- `internal/config/config.go` (MODIFY)

- internal/session/session.go (MODIFY)
- internal/llm/providers/ (NEW directory)
  - internal/llm/providers/openai.go
  - internal/llm/providers/anthropic.go
  - internal/llm/providers/gemini.go
  - internal/llm/providers/ollama.go
  - internal/llm/providers/bedrock.go
  - internal/llm/providers/openrouter.go
  - internal/llm/providers/groq.go
  - internal/llm/providers/azure.go
  - internal/llm/providers/copilot.go
  - internal/llm/providers/local.go

Exact Code Changes

NEW FILE: internal/llm/model_descriptor.go

```
package llm

import (
    "encoding/json"
    "fmt"
    "net/url"
    "time"

    "github.com/hashicorp/golang-lru/v2/expirable"
)

// ModelCapability describes what a model can do
type ModelCapability string

const (
    CapCodeGen    ModelCapability = "code-gen"
    CapCodeEdit   ModelCapability = "code-edit"
    CapReasoning  ModelCapability = "reasoning"
    CapVision     ModelCapability = "vision"
    CapAudio      ModelCapability = "audio"
    CapMultimodal ModelCapability = "multimodal"
    CapLongContext ModelCapability = "long-context"
    CapToolUse    ModelCapability = "tool-use"
    CapJSONMode   ModelCapability = "json-mode"
)

// ModelDescriptor is a canonical model description (mirrors
        models.dev schema)
type ModelDescriptor struct {
    ID               string            `json:"id" yaml:"id"`
    Name             string            `json:"name" yaml:"name"`
    Provider         string            `json:"provider"
        yaml:"provider"`
```

```
    ContextSize      int                `json:"context_size"
        yaml:"context_size"`
    MaxOutputTokens  int                `json:"max_output_tokens"
        yaml:"max_output_tokens"`
    InputPricePer1K  float64
        `json:"input_price_per_1k"
        yaml:"input_price_per_1k"`   // USD per 1K tokens
    OutputPricePer1K float64
        `json:"output_price_per_1k"
        yaml:"output_price_per_1k"` // USD per 1K tokens
    Capabilities     []ModelCapability `json:"capabilities"
        yaml:"capabilities"`
    DefaultTemp      float64           `json:"default_temp"
        yaml:"default_temp"`
    SupportsStreaming bool
        `json:"supports_streaming" yaml:"supports_streaming"`
    SupportsVision   bool              `json:"supports_vision"
        yaml:"supports_vision"`
    SupportsTools    bool              `json:"supports_tools"
        yaml:"supports_tools"`
    IsLocal          bool              `json:"is_local"
        yaml:"is_local"`
    Endpoint         *url.URL          `json:"-" yaml:"-"`
    APIKeyEnv        string            `json:"api_key_env"
        yaml:"api_key_env"`
    APIKey           string            `json:"-" yaml:"-"`
    CreatedAt        time.Time         `json:"created_at"
        yaml:"created_at"`
    UpdatedAt        time.Time         `json:"updated_at"
        yaml:"updated_at"`
    LatencyMs        float64
        `json:"latency_ms,omitempty" yaml:"latency_ms,omitempty"`
    ErrorRate        float64
        `json:"error_rate,omitempty" yaml:"error_rate,omitempty"`
}

// CostEstimate returns estimated cost for a given token count
func (m *ModelDescriptor) CostEstimate(inputTokens, outputTokens
        int) float64 {
    inputCost := float64(inputTokens) / 1000.0 *
        m.InputPricePer1K
    outputCost := float64(outputTokens) / 1000.0 *
        m.OutputPricePer1K
    return inputCost + outputCost
}

// IsCapable returns true if model has the given capability
func (m *ModelDescriptor) IsCapable(cap ModelCapability) bool {
```

```
    for _, c := range m.Capabilities {
        if c == cap {
            return true
        }
    }
    return false
}

// String returns a human-readable summary
func (m *ModelDescriptor) String() string {
    return fmt.Sprintf("%s@%s (ctx=%d, vision=%v, tools=%v,
        local=%v)",
        m.ID, m.Provider, m.ContextSize, m.SupportsVision,
        m.SupportsTools, m.IsLocal)
}

// ModelCache is an LRU cache for model descriptors (from
        models.dev / local registry)
type ModelCache struct {
    cache *expirable.LRU[string, *ModelDescriptor]
}

func NewModelCache() *ModelCache {
    return &ModelCache{
        cache: expirable.NewLRU[string, *ModelDescriptor](500,
        nil, time.Hour),
    }
}

func (mc *ModelCache) Get(key string) (*ModelDescriptor, bool) {
    return mc.cache.Get(key)
}

func (mc *ModelCache) Add(key string, m *ModelDescriptor) {
    mc.cache.Add(key, m)
}

// ProviderKind is an enum for supported backends
type ProviderKind string

const (
    ProviderOpenAI     ProviderKind = "openai"
    ProviderAnthropic  ProviderKind = "anthropic"
    ProviderGemini     ProviderKind = "gemini"
    ProviderOllama     ProviderKind = "ollama"
    ProviderBedrock    ProviderKind = "bedrock"
    ProviderOpenRouter ProviderKind = "openrouter"
    ProviderGroq       ProviderKind = "groq"
```

```
    ProviderAzure      ProviderKind = "azure"
    ProviderCopilot    ProviderKind = "copilot"
    ProviderLocal      ProviderKind = "local"
    ProviderCustom     ProviderKind = "custom"
)
```

NEW FILE: `internal/llm/provider_router.go`

```
package llm

import (
    "context"
    "errors"
    "fmt"
    "log"
    "sync"
    "time"
)

// ErrNoHealthyProvider is returned when all providers are
        unhealthy
var ErrNoHealthyProvider = errors.New("no healthy providers
        available")

// ErrProviderNotFound is returned when a requested provider is
        not registered
var ErrProviderNotFound = errors.New("provider not found")

// RouterStrategy defines how to pick a provider
type RouterStrategy string

const (
    StrategyCheapest    RouterStrategy = "cheapest"
    StrategyFastest     RouterStrategy = "fastest"
    StrategyMostCapable RouterStrategy = "most_capable"
    StrategyRoundRobin  RouterStrategy = "round_robin"
    StrategyExplicit    RouterStrategy = "explicit"
    StrategyFallback    RouterStrategy = "fallback"
)

// ProviderRouter dynamically routes LLM requests across 75+
        providers
type ProviderRouter struct {
    providers      map[ProviderKind]Provider
    healthMonitor  *HealthMonitor
    strategy       RouterStrategy
    registry       *ProviderRegistry
    sessionModel   map[string]string // sessionID -> modelID
```

```
    sessionMu       sync.RWMutex
    roundRobinIdx   int
    mu              sync.RWMutex
}

func NewProviderRouter(registry *ProviderRegistry, strategy
        RouterStrategy) *ProviderRouter {
    return &ProviderRouter{
        providers:     make(map[ProviderKind]Provider),
        healthMonitor: NewHealthMonitor(time.Minute),
        strategy:      strategy,
        registry:      registry,
        sessionModel:  make(map[string]string),
    }
}

// RegisterProvider adds a provider to the router
func (pr *ProviderRouter) RegisterProvider(kind ProviderKind, p
        Provider) error {
    pr.mu.Lock()
    defer pr.mu.Unlock()

    if _, exists := pr.providers[kind]; exists {
        return fmt.Errorf("provider %s already registered", kind)
    }

    pr.providers[kind] = p
    pr.healthMonitor.Register(kind)
    log.Printf("[ProviderRouter] Registered provider: %s", kind)
    return nil
}

// UnregisterProvider removes a provider from the router
func (pr *ProviderRouter) UnregisterProvider(kind ProviderKind) {
    pr.mu.Lock()
    defer pr.mu.Unlock()
    delete(pr.providers, kind)
    pr.healthMonitor.Unregister(kind)
}

// SetSessionModel locks a session to a specific model (mid-
        session switching)
func (pr *ProviderRouter) SetSessionModel(sessionID, modelID
        string) error {
    pr.sessionMu.Lock()
    defer pr.sessionMu.Unlock()

    // Verify model exists in registry
```

```
    if _, err := pr.registry.GetModel(modelID); err != nil {
        return fmt.Errorf("cannot set session model: %w", err)
    }
    pr.sessionModel[sessionID] = modelID
    log.Printf("[ProviderRouter] Session %s -> model %s",
        sessionID, modelID)
    return nil
}

// GetSessionModel returns the model ID locked for this session
func (pr *ProviderRouter) GetSessionModel(sessionID string)
        (string, bool) {
    pr.sessionMu.RLock()
    defer pr.sessionMu.RUnlock()
    m, ok := pr.sessionModel[sessionID]
    return m, ok
}

// ClearSessionModel removes the model lock for a session
func (pr *ProviderRouter) ClearSessionModel(sessionID string) {
    pr.sessionMu.Lock()
    defer pr.sessionMu.Unlock()
    delete(pr.sessionModel, sessionID)
}

// Route selects a provider based on strategy + requirements +
        health
func (pr *ProviderRouter) Route(
    ctx context.Context,
    sessionID string,
    requiredCaps []ModelCapability,
    preferredModel string,
) (Provider, *ModelDescriptor, error) {

    // 1. Check if session has an explicit model lock
    if sessionModel, ok := pr.GetSessionModel(sessionID); ok {
        preferredModel = sessionModel
    }

    // 2. If explicit model requested, resolve it directly
    if preferredModel != "" {
        model, err := pr.registry.GetModel(preferredModel)
        if err != nil {
            return nil, nil, err
        }
        providerKind := ProviderKind(model.Provider)
        if p, ok := pr.getProvider(providerKind); ok {
            if pr.healthMonitor.IsHealthy(providerKind) {
```

```
                return p, model, nil
            }
            log.Printf("[ProviderRouter] Explicit provider %s
        unhealthy, falling back", providerKind)
        }
        // If explicit provider is unhealthy, fall through to
        strategy
    }

    // 3. Gather all healthy providers that support required
        capabilities
    candidates := pr.filterHealthyByCapabilities(requiredCaps)
    if len(candidates) == 0 {
        return nil, nil, ErrNoHealthyProvider
    }

    // 4. Apply strategy
    switch pr.strategy {
    case StrategyCheapest:
        return pr.selectCheapest(candidates)
    case StrategyFastest:
        return pr.selectFastest(candidates)
    case StrategyMostCapable:
        return pr.selectMostCapable(candidates, requiredCaps)
    case StrategyRoundRobin:
        return pr.selectRoundRobin(candidates)
    case StrategyFallback:
        return pr.selectFallback(candidates)
    default:
        return pr.selectCheapest(candidates)
    }
}

func (pr *ProviderRouter) getProvider(kind ProviderKind)
        (Provider, bool) {
    pr.mu.RLock()
    defer pr.mu.RUnlock()
    p, ok := pr.providers[kind]
    return p, ok
}

// candidate is a provider + model combo ready for selection
type candidate struct {
    provider Provider
    model    *ModelDescriptor
    score    float64
}
```

```
func (pr *ProviderRouter) filterHealthyByCapabilities(
    requiredCaps []ModelCapability,
) []candidate {
    pr.mu.RLock()
    defer pr.mu.RUnlock()

    var results []candidate
    for kind, p := range pr.providers {
        if !pr.healthMonitor.IsHealthy(kind) {
            continue
        }
        models := p.GetModels()
        for _, m := range models {
            if pr.modelSupportsCaps(m, requiredCaps) {
                results = append(results, candidate{provider: p,
        model: m})
            }
        }
    }
    return results
}

func (pr *ProviderRouter) modelSupportsCaps(m *ModelDescriptor,
        caps []ModelCapability) bool {
    for _, cap := range caps {
        if !m.IsCapable(cap) {
            return false
        }
    }
    return true
}

func (pr *ProviderRouter) selectCheapest(candidates []candidate)
        (Provider, *ModelDescriptor, error) {
    var best candidate
    found := false
    for _, c := range candidates {
        score := c.model.InputPricePer1K +
        c.model.OutputPricePer1K
        if !found || score < best.score {
            best = candidate{c.provider, c.model, score}
            found = true
        }
    }
    if !found {
        return nil, nil, ErrNoHealthyProvider
    }
    return best.provider, best.model, nil
```

```
}

func (pr *ProviderRouter) selectFastest(candidates []candidate)
        (Provider, *ModelDescriptor, error) {
    var best candidate
    found := false
    for _, c := range candidates {
        score := c.model.LatencyMs
        if !found || score < best.score {
            best = candidate{c.provider, c.model, score}
            found = true
        }
    }
    if !found {
        return nil, nil, ErrNoHealthyProvider
    }
    return best.provider, best.model, nil
}

func (pr *ProviderRouter) selectMostCapable(
    candidates []candidate,
    requiredCaps []ModelCapability,
) (Provider, *ModelDescriptor, error) {
    var best candidate
    found := false
    for _, c := range candidates {
        score := float64(len(c.model.Capabilities))
        if !found || score > best.score {
            best = candidate{c.provider, c.model, score}
            found = true
        }
    }
    if !found {
        return nil, nil, ErrNoHealthyProvider
    }
    return best.provider, best.model, nil
}

func (pr *ProviderRouter) selectRoundRobin(candidates
        []candidate) (Provider, *ModelDescriptor, error) {
    pr.mu.Lock()
    idx := pr.roundRobinIdx
    pr.roundRobinIdx = (pr.roundRobinIdx + 1) % len(candidates)
    pr.mu.Unlock()
    if len(candidates) == 0 {
        return nil, nil, ErrNoHealthyProvider
    }
    c := candidates[idx%len(candidates)]
```

```
    return c.provider, c.model, nil
}

func (pr *ProviderRouter) selectFallback(candidates []candidate)
        (Provider, *ModelDescriptor, error) {
    // Use first available, then next on failure
    for _, c := range candidates {
        return c.provider, c.model, nil
    }
    return nil, nil, ErrNoHealthyProvider
}

// ListAllModels returns all models across all registered
        providers
func (pr *ProviderRouter) ListAllModels() []*ModelDescriptor {
    pr.mu.RLock()
    defer pr.mu.RUnlock()

    var all []*ModelDescriptor
    seen := make(map[string]bool)
    for _, p := range pr.providers {
        for _, m := range p.GetModels() {
            if !seen[m.ID] {
                all = append(all, m)
                seen[m.ID] = true
            }
        }
    }
    return all
}

// SwitchModel performs mid-session model switching
func (pr *ProviderRouter) SwitchModel(
    ctx context.Context,
    sessionID string,
    newModelID string,
) (*ModelDescriptor, error) {
    oldModel, hadOld := pr.GetSessionModel(sessionID)

    if err := pr.SetSessionModel(sessionID, newModelID); err !=
        nil {
        return nil, fmt.Errorf("failed to switch model: %w", err)
    }

    newModel, err := pr.registry.GetModel(newModelID)
    if err != nil {
        if hadOld {
            pr.SetSessionModel(sessionID, oldModel)
```

```
        } else {
            pr.ClearSessionModel(sessionID)
        }
        return nil, err
    }

    log.Printf("[ProviderRouter] Model switch: %s -> %s
        (session=%s)", oldModel, newModelID, sessionID)
    return newModel, nil
}
```

NEW FILE: internal/llm/health_monitor.go

```
package llm

import (
    "context"
    "fmt"
    "log"
    "sync"
    "time"
)

// ProviderHealth tracks the health of a single provider
type ProviderHealth struct {
    Kind           ProviderKind
    LastCheck      time.Time
    Status         HealthStatus
    Latency        time.Duration
    ErrorCount     int
    SuccessCount   int
    ConsecutiveErr int
    LastError      string
}

// HealthStatus enum
type HealthStatus string

const (
    HealthUnknown   HealthStatus = "unknown"
    HealthHealthy   HealthStatus = "healthy"
    HealthDegraded  HealthStatus = "degraded"
    HealthUnhealthy HealthStatus = "unhealthy"
)

// HealthMonitor periodically checks provider health
type HealthMonitor struct {
    checkInterval time.Duration
```

```
    healths       map[ProviderKind]*ProviderHealth
    mu            sync.RWMutex
    stopCh        chan struct{}
    wg            sync.WaitGroup
}

func NewHealthMonitor(interval time.Duration) *HealthMonitor {
    return &HealthMonitor{
        checkInterval: interval,
        healths:       make(map[ProviderKind]*ProviderHealth),
        stopCh:        make(chan struct{}),
    }
}

// Register a provider for health monitoring
func (hm *HealthMonitor) Register(kind ProviderKind) {
    hm.mu.Lock()
    defer hm.mu.Unlock()
    hm.healths[kind] = &ProviderHealth{
        Kind:   kind,
        Status: HealthUnknown,
    }
}

// Unregister a provider from health monitoring
func (hm *HealthMonitor) Unregister(kind ProviderKind) {
    hm.mu.Lock()
    defer hm.mu.Unlock()
    delete(hm.healths, kind)
}

// IsHealthy returns true if the provider is healthy or degraded
func (hm *HealthMonitor) IsHealthy(kind ProviderKind) bool {
    hm.mu.RLock()
    defer hm.mu.RUnlock()
    h, ok := hm.healths[kind]
    if !ok {
        return false
    }
    return h.Status == HealthHealthy || h.Status ==
        HealthDegraded
}

// GetHealth returns the health status of a provider
func (hm *HealthMonitor) GetHealth(kind ProviderKind)
        (*ProviderHealth, bool) {
    hm.mu.RLock()
    defer hm.mu.RUnlock()
```

```
    h, ok := hm.healths[kind]
    return h, ok
}

// Start begins periodic health checks
func (hm *HealthMonitor) Start(ctx context.Context) {
    ticker := time.NewTicker(hm.checkInterval)
    defer ticker.Stop()

    for {
        select {
        case <-ctx.Done():
            return
        case <-ticker.C:
            hm.runChecks()
        case <-hm.stopCh:
            return
        }
    }
}

// Stop halts the health monitor
func (hm *HealthMonitor) Stop() {
    close(hm.stopCh)
    hm.wg.Wait()
}

func (hm *HealthMonitor) runChecks() {
    hm.mu.RLock()
    providers := make([]ProviderKind, 0, len(hm.healths))
    for k := range hm.healths {
        providers = append(providers, k)
    }
    hm.mu.RUnlock()

    for _, kind := range providers {
        hm.wg.Add(1)
        go func(k ProviderKind) {
            defer hm.wg.Done()
            hm.checkProvider(k)
        }(kind)
    }
}

func (hm *HealthMonitor) checkProvider(kind ProviderKind) {
    // In real implementation, this performs a lightweight LLM
        call or status check
    // For now, simulate with configurable hooks
```

```
    start := time.Now()

    // Attempt a cheap health probe (e.g., models list,
        lightweight completion)
    // This is a placeholder - actual implementation calls
        provider.HealthCheck()

    latency := time.Since(start)

    hm.mu.Lock()
    defer hm.mu.Unlock()
    h, ok := hm.healths[kind]
    if !ok {
        return
    }

    h.LastCheck = time.Now()
    h.Latency = latency

    // Update status based on latency + error rate thresholds
    switch {
    case h.ConsecutiveErr >= 5:
        h.Status = HealthUnhealthy
    case h.ConsecutiveErr >= 2:
        h.Status = HealthDegraded
    case latency > 30*time.Second:
        h.Status = HealthDegraded
    default:
        h.Status = HealthHealthy
    }
}

// RecordSuccess records a successful call
func (hm *HealthMonitor) RecordSuccess(kind
        ProviderKind, latency time.Duration) {
    hm.mu.Lock()
    defer hm.mu.Unlock()
    h, ok := hm.healths[kind]
    if !ok {
        return
    }
    h.SuccessCount++
    h.ConsecutiveErr = 0
    if latency > 0 {
        h.Latency = latency
    }
}
```

```go
// RecordFailure records a failed call
func (hm *HealthMonitor) RecordFailure(kind ProviderKind, err
        error) {
    hm.mu.Lock()
    defer hm.mu.Unlock()
    h, ok := hm.healths[kind]
    if !ok {
        return
    }
    h.ErrorCount++
    h.ConsecutiveErr++
    h.LastError = err.Error()

    if h.ConsecutiveErr >= 5 {
        h.Status = HealthUnhealthy
    } else if h.ConsecutiveErr >= 2 {
        h.Status = HealthDegraded
    }
}

// GetAllHealth returns health of all providers
func (hm *HealthMonitor) GetAllHealth() []*ProviderHealth {
    hm.mu.RLock()
    defer hm.mu.RUnlock()
    results := make([]*ProviderHealth, 0, len(hm.healths))
    for _, h := range hm.healths {
        // Copy to avoid data race
        results = append(results, &ProviderHealth{
            Kind:           h.Kind,
            LastCheck:      h.LastCheck,
            Status:         h.Status,
            Latency:        h.Latency,
            ErrorCount:     h.ErrorCount,
            SuccessCount:   h.SuccessCount,
            ConsecutiveErr: h.ConsecutiveErr,
            LastError:      h.LastError,
        })
    }
    return results
}
```

NEW FILE: `internal/llm/provider_registry.go`

```go
package llm

import (
    "encoding/json"
    "fmt"
```

```
    "io"
    "net/http"
    "os"
    "sync"
    "time"
)

const modelsDevCatalogURL = "https://api.models.dev/v1/models"

// ProviderRegistry holds the canonical model catalog
type ProviderRegistry struct {
    models    map[string]*ModelDescriptor
    mu        sync.RWMutex
    cache     *ModelCache
    client    *http.Client
    localPath string
}

func NewProviderRegistry(localPath string) *ProviderRegistry {
    return &ProviderRegistry{
        models:    make(map[string]*ModelDescriptor),
        cache:     NewModelCache(),
        client:    &http.Client{Timeout: 30 * time.Second},
        localPath: localPath,
    }
}

// LoadLocalModels loads user-defined models from ~/.helix/
        models.json
func (reg *ProviderRegistry) LoadLocalModels() error {
    data, err := os.ReadFile(reg.localPath)
    if err != nil {
        if os.IsNotExist(err) {
            return nil
        }
        return err
    }

    var models []*ModelDescriptor
    if err := json.Unmarshal(data, &models); err != nil {
        return fmt.Errorf("failed to parse local models: %w",
        err)
    }

    reg.mu.Lock()
    defer reg.mu.Unlock()
    for _, m := range models {
        reg.models[m.ID] = m
```

```
    }
    return nil
}

// FetchRemoteCatalog fetches models from models.dev or similar
        service
func (reg *ProviderRegistry) FetchRemoteCatalog(ctx
        context.Context) error {
    req, err := http.NewRequestWithContext(ctx, "GET",
        modelsDevCatalogURL, nil)
    if err != nil {
        return err
    }

    resp, err := reg.client.Do(req)
    if err != nil {
        return fmt.Errorf("models.dev unreachable: %w", err)
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return fmt.Errorf("models.dev returned %d",
        resp.StatusCode)
    }

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        return err
    }

    var remoteModels struct {
        Models []*ModelDescriptor `json:"models"`
    }
    if err := json.Unmarshal(body, &remoteModels); err != nil {
        return fmt.Errorf("failed to parse models.dev response:
        %w", err)
    }

    reg.mu.Lock()
    defer reg.mu.Unlock()
    for _, m := range remoteModels.Models {
        m.UpdatedAt = time.Now()
        reg.models[m.ID] = m
        reg.cache.Add(m.ID, m)
    }

    return nil
}
```

```
// GetModel returns a model by ID
func (reg *ProviderRegistry) GetModel(id string)
        (*ModelDescriptor, error) {
    reg.mu.RLock()
    m, ok := reg.models[id]
    reg.mu.RUnlock()
    if ok {
        return m, nil
    }

    // Try cache
    if cached, ok := reg.cache.Get(id); ok {
        return cached, nil
    }

    return nil, fmt.Errorf("model %s not found", id)
}

// GetModelsByProvider returns all models for a provider
func (reg *ProviderRegistry) GetModelsByProvider(provider
        string) []*ModelDescriptor {
    reg.mu.RLock()
    defer reg.mu.RUnlock()

    var result []*ModelDescriptor
    for _, m := range reg.models {
        if m.Provider == provider {
            result = append(result, m)
        }
    }
    return result
}

// GetModelsByCapability returns models supporting a given
        capability
func (reg *ProviderRegistry) GetModelsByCapability(cap
        ModelCapability) []*ModelDescriptor {
    reg.mu.RLock()
    defer reg.mu.RUnlock()

    var result []*ModelDescriptor
    for _, m := range reg.models {
        if m.IsCapable(cap) {
            result = append(result, m)
        }
    }
    return result
}
```

```
// SaveLocalModels persists the current model list to local file
func (reg *ProviderRegistry) SaveLocalModels() error {
    reg.mu.RLock()
    models := make([]*ModelDescriptor, 0, len(reg.models))
    for _, m := range reg.models {
        models = append(models, m)
    }
    reg.mu.RUnlock()

    data, err := json.MarshalIndent(models, "", "  ")
    if err != nil {
        return err
    }

    return os.WriteFile(reg.localPath, data, 0644)
}

// AddModel adds a custom model to the registry
func (reg *ProviderRegistry) AddModel(m *ModelDescriptor) error {
    if m.ID == "" || m.Provider == "" {
        return fmt.Errorf("model ID and provider are required")
    }
    reg.mu.Lock()
    defer reg.mu.Unlock()
    m.CreatedAt = time.Now()
    m.UpdatedAt = time.Now()
    reg.models[m.ID] = m
    return nil
}

// RemoveModel removes a model from the registry
func (reg *ProviderRegistry) RemoveModel(id string) {
    reg.mu.Lock()
    defer reg.mu.Unlock()
    delete(reg.models, id)
}

// ListAll returns all registered models
func (reg *ProviderRegistry) ListAll() []*ModelDescriptor {
    reg.mu.RLock()
    defer reg.mu.RUnlock()
    result := make([]*ModelDescriptor, 0, len(reg.models))
    for _, m := range reg.models {
        result = append(result, m)
    }
    return result
}
```

MODIFY: `internal/llm/llm.go` **(existing interface augmentation)**

```
// In internal/llm/llm.go, add to the existing Provider
        interface:

type Provider interface {
    // Existing methods
    Generate(ctx context.Context, prompt string,
        opts ...GenerateOption) (string, error)
    GenerateStream(ctx context.Context, prompt string,
        opts ...GenerateOption) (<-chan StreamChunk, error)
    GetModels() []*ModelDescriptor

    // NEW: Health check probe
    HealthCheck(ctx context.Context) (time.Duration, error)

    // NEW: Model capabilities
    SupportsModel(modelID string) bool

    // NEW: Cost estimation
    EstimateCost(inputTokens, outputTokens int) float64
}

// StreamChunk is an existing type; keep it
```

MODIFY: `cmd/cli/main.go` **— Add model switching command**

```
// Add to CLI struct in cmd/cli/main.go:

type CLI struct {
    // ... existing fields ...
    providerRouter *llm.ProviderRouter
    providerReg    *llm.ProviderRegistry
}

// In NewCLI(), initialize:
func NewCLI() *CLI {
    // ... existing initialization ...

    // NEW: Initialize provider registry + router
    providerReg :=
        llm.NewProviderRegistry(os.ExpandEnv("$HOME/.helix/
        models.json"))
    _ = providerReg.LoadLocalModels()

    router := llm.NewProviderRouter(providerReg,
        llm.StrategyFallback)
```

```
    // Register built-in providers from config
    cfg, _ := config.Load()
    if cfg != nil {
        if oa := cfg.OpenAI; oa != nil && oa.APIKey != "" {
            p, _ := llm.NewOpenAIProvider(oa)
            router.RegisterProvider(llm.ProviderOpenAI, p)
        }
        if cl := cfg.Anthropic; cl != nil && cl.APIKey != "" {
            p, _ := llm.NewAnthropicProvider(cl)
            router.RegisterProvider(llm.ProviderAnthropic, p)
        }
        // ... etc for all configured providers
    }

    // Start health monitor
    go router.healthMonitor.Start(ctx)

    return &CLI{
        // ... existing fields ...
        providerRouter: router,
        providerReg:    providerReg,
    }
}

// Add flag in Run():
modelSwitch := flag.String("model-switch", "", "Switch model mid-
        session (format: sessionID:modelID)")

// Add case in switch:
case *modelSwitch != "":
    parts := strings.SplitN(*modelSwitch, ":", 2)
    if len(parts) != 2 {
        return fmt.Errorf("model-switch format must be
        sessionID:modelID")
    }
    return c.handleModelSwitch(ctx, parts[0], parts[1])

// Add handler:
func (c *CLI) handleModelSwitch(ctx context.Context, sessionID,
        modelID string) error {
    md, err := c.providerRouter.SwitchModel(ctx, sessionID,
        modelID)
    if err != nil {
        return err
    }
    fmt.Printf("Switched session %s to model: %s\n", sessionID,
        md.String())
```

```
    return nil
}
```

NEW FILE: internal/llm/providers/openai.go

```
package providers

import (
    "context"
    "fmt"
    "time"

    "dev.helix.code/internal/llm"
)

type OpenAIProvider struct {
    apiKey      string
    baseURL     string
    models      []*llm.ModelDescriptor
    healthy     bool
}

func NewOpenAIProvider(cfg llm.OpenAIConfig) (*OpenAIProvider,
        error) {
    p := &OpenAIProvider{
        apiKey:  cfg.APIKey,
        baseURL: cfg.BaseURL,
        models:  make([]*llm.ModelDescriptor, 0),
    }
    // Register known OpenAI models
    p.models = append(p.models, &llm.ModelDescriptor{
        ID: "gpt-4o", Name: "GPT-4o", Provider: "openai",
        ContextSize: 128000, MaxOutputTokens: 16384,
        InputPricePer1K: 5.0, OutputPricePer1K: 15.0,
        Capabilities: []llm.ModelCapability{llm.CapCodeGen,
         llm.CapCodeEdit, llm.CapVision, llm.CapToolUse},
        SupportsVision: true, SupportsTools: true,
    })
    p.models = append(p.models, &llm.ModelDescriptor{
        ID: "gpt-4o-mini", Name: "GPT-4o Mini", Provider:
         "openai",
        ContextSize: 128000, MaxOutputTokens: 16384,
        InputPricePer1K: 0.15, OutputPricePer1K: 0.6,
        Capabilities: []llm.ModelCapability{llm.CapCodeGen,
         llm.CapToolUse},
        SupportsTools: true,
    })
    // ... more models
```

```
    return p, nil
}

func (p *OpenAIProvider) Generate(ctx context.Context, prompt
        string, opts ...llm.GenerateOption) (string, error) {
    // Delegate to existing OpenAI implementation or new client
    return "", fmt.Errorf("not implemented: delegate to internal/
        llm/openai.go")
}

func (p *OpenAIProvider) GenerateStream(ctx context.Context,
        prompt string, opts ...llm.GenerateOption) (<-chan
        llm.StreamChunk, error) {
    return nil, fmt.Errorf("not implemented")
}

func (p *OpenAIProvider) GetModels() []*llm.ModelDescriptor {
    return p.models
}

func (p *OpenAIProvider) HealthCheck(ctx context.Context)
        (time.Duration, error) {
    start := time.Now()
    // Perform lightweight models list call
    return time.Since(start), nil
}

func (p *OpenAIProvider) SupportsModel(modelID string) bool {
    for _, m := range p.models {
        if m.ID == modelID {
            return true
        }
    }
    return false
}

func (p *OpenAIProvider) EstimateCost(inputTokens, outputTokens
        int) float64 {
    // Default: delegate to first model's pricing
    if len(p.models) > 0 {
        return p.models[0].CostEstimate(inputTokens,
        outputTokens)
    }
    return 0
}
```

Anti-Bluff Test

```
# TEST 1: Verify provider registration
cd /mnt/agents/output/helixcode_sources
go test ./internal/llm/ -run TestProviderRouter_Register -v

# TEST 2: Verify model switching
go test ./internal/llm/ -run TestProviderRouter_SwitchModel -v

# TEST 3: Verify health monitoring
go test ./internal/llm/ -run TestHealthMonitor -v

# TEST 4: End-to-end: start a session with one model, switch mid-
        session, verify context preservation
go test ./internal/session/ -run TestMidSessionModelSwitch -v

# TEST 5: Verify 75+ model descriptors loaded
go test ./internal/llm/ -run
        TestProviderRegistry_Load75PlusModels -v
```

Integration Verification

- [ ] `internal/llm/provider_router.go` compiles with `go build ./internal/llm/`
- [ ] All provider implementations implement the `llm.Provider` interface
- [ ] Health monitor goroutine starts and stops cleanly
- [ ] Model switching updates session metadata in PostgreSQL
- [ ] CLI `--model-switch sessionID:modelID` works end-to-end
- [ ] No memory leaks in provider cache (LRU eviction confirmed)

Feature 2: LSP Integration with Diagnostics Feedback

Source Location (in original agent)

- `opencode/src/lsp/client.ts` — LSP client wrapper
- `opencode/src/lsp/server-manager.ts` — Language server lifecycle
- `opencode/src/lsp/diagnostics.ts` — Diagnostics collection
- `opencode/src/tools/diagnostics.ts` — AI diagnostics tool
- `opencode/src/lsp/protocol.ts` — LSP JSON-RPC protocol

Target Location (in HelixCode)

- `internal/tools/lsp_client.go` (NEW)
- `internal/tools/lsp_server_manager.go` (NEW)
- `internal/tools/lsp_diagnostics.go` (NEW)
- `internal/tools/lsp_protocol.go` (NEW)
- `internal/tools/diagnostics_tool.go` (NEW)
- `internal/config/config.go` (MODIFY)
- `internal/session/session.go` (MODIFY)
- `internal/llm/prompt_builder.go` (MODIFY)

Exact Code Changes

NEW FILE: `internal/tools/lsp_protocol.go`

```
package tools

import (
    "encoding/json"
    "fmt"
    "strconv"
)

// LSPMessage is the base JSON-RPC envelope
type LSPMessage struct {
    JSONRPC string          `json:"jsonrpc"`
    ID      interface{}     `json:"id,omitempty"`
    Method  string          `json:"method,omitempty"`
    Params  json.RawMessage `json:"params,omitempty"`
    Result  json.RawMessage `json:"result,omitempty"`
    Error   *LSPError       `json:"error,omitempty"`
}

type LSPError struct {
    Code    int    `json:"code"`
    Message string `json:"message"`
    Data    interface{} `json:"data,omitempty"`
}

// InitializeParams (LSP spec)
type InitializeParams struct {
    ProcessID int    `json:"processId"`
    RootURI   string `json:"rootUri"`
    RootPath  string `json:"rootPath,omitempty"`
    Capabilities ClientCapabilities `json:"capabilities"`
}

type ClientCapabilities struct {
```

```
    Workspace    WorkspaceClientCapabilities
        `json:"workspace,omitempty"`
    TextDocument TextDocumentClientCapabilities
        `json:"textDocument,omitempty"`
}

type WorkspaceClientCapabilities struct {
    ApplyEdit bool `json:"applyEdit,omitempty"`
}

type TextDocumentClientCapabilities struct {
    PublishDiagnostics PublishDiagnosticsCapabilities
        `json:"publishDiagnostics,omitempty"`
}

type PublishDiagnosticsCapabilities struct {
    RelatedInformation bool `json:"relatedInformation,omitempty"`
    VersionSupport     bool `json:"versionSupport,omitempty"`
}

type InitializeResult struct {
    Capabilities ServerCapabilities `json:"capabilities"`
}

type ServerCapabilities struct {
    TextDocumentSync   *TextDocumentSyncOptions
        `json:"textDocumentSync,omitempty"`
    HoverProvider      bool
        `json:"hoverProvider,omitempty"`
    CompletionProvider *CompletionOptions
        `json:"completionProvider,omitempty"`
    DefinitionProvider bool
        `json:"definitionProvider,omitempty"`
    DiagnosticProvider *DiagnosticOptions
        `json:"diagnosticProvider,omitempty"`
}

type TextDocumentSyncOptions struct {
    OpenClose bool `json:"openClose,omitempty"`
    Change    int  `json:"change,omitempty"` // 0=None, 1=Full,
        2=Incremental
}

type CompletionOptions struct {
    TriggerCharacters []string
        `json:"triggerCharacters,omitempty"`
}

type DiagnosticOptions struct {
```

```
    Identifier            string `json:"identifier,omitempty"`
    InterFileDependencies bool
        `json:"interFileDependencies,omitempty"`
    WorkspaceDiagnostics  bool
        `json:"workspaceDiagnostics,omitempty"`
}

// DidOpenTextDocumentNotification
type DidOpenTextDocumentParams struct {
    TextDocument TextDocumentItem `json:"textDocument"`
}

type TextDocumentItem struct {
    URI        string `json:"uri"`
    LanguageID string `json:"languageId"`
    Version    int    `json:"version"`
    Text       string `json:"text"`
}

// DidChangeTextDocumentNotification
type DidChangeTextDocumentParams struct {
    TextDocument   VersionedTextDocumentIdentifier
        `json:"textDocument"`
    ContentChanges []TextDocumentContentChangeEvent
        `json:"contentChanges"`
}

type VersionedTextDocumentIdentifier struct {
    URI     string `json:"uri"`
    Version int    `json:"version"`
}

type TextDocumentContentChangeEvent struct {
    Text string `json:"text"`
}

// Diagnostic-related structures
type PublishDiagnosticsParams struct {
    URI         string       `json:"uri"`
    Version     int          `json:"version,omitempty"`
    Diagnostics []Diagnostic `json:"diagnostics"`
}

type Diagnostic struct {
    Range    Range              `json:"range"`
    Severity DiagnosticSeverity `json:"severity,omitempty"`
    Code     interface{}        `json:"code,omitempty"`
    Source   string             `json:"source,omitempty"`
```

```
    Message  string              `json:"message"`
    RelatedInformation []DiagnosticRelatedInformation
        `json:"relatedInformation,omitempty"`
}

type Range struct {
    Start Position `json:"start"`
    End   Position `json:"end"`
}

type Position struct {
    Line      int `json:"line"`
    Character int `json:"character"`
}

type DiagnosticSeverity int

const (
    SeverityError       DiagnosticSeverity = 1
    SeverityWarning     DiagnosticSeverity = 2
    SeverityInformation DiagnosticSeverity = 3
    SeverityHint        DiagnosticSeverity = 4
)

type DiagnosticRelatedInformation struct {
    Location Location `json:"location"`
    Message  string   `json:"message"`
}

type Location struct {
    URI   string `json:"uri"`
    Range Range  `json:"range"`
}

// DocumentDiagnosticParams for pull diagnostics (LSP 3.17+)
type DocumentDiagnosticParams struct {
    TextDocument TextDocumentIdentifier `json:"textDocument"`
}

type TextDocumentIdentifier struct {
    URI string `json:"uri"`
}

type DocumentDiagnosticReport struct {
    Kind       string       `json:"kind"` // "full" or
        "unchanged"
    ResultID   string       `json:"resultId,omitempty"`
    Items      []Diagnostic `json:"items,omitempty"`
}
```

```go
// Helper: Build JSON-RPC request
func NewRequest(id int, method string, params interface{})
        (*LSPMessage, []byte, error) {
    paramsBytes, err := json.Marshal(params)
    if err != nil {
        return nil, nil, err
    }
    msg := &LSPMessage{
        JSONRPC: "2.0",
        ID:      id,
        Method:  method,
        Params:  paramsBytes,
    }
    data, err := json.Marshal(msg)
    if err != nil {
        return nil, nil, err
    }
    return msg, data, nil
}

// Helper: Build JSON-RPC notification (no ID)
func NewNotification(method string, params interface{}) ([]byte,
        error) {
    paramsBytes, err := json.Marshal(params)
    if err != nil {
        return nil, err
    }
    msg := &LSPMessage{
        JSONRPC: "2.0",
        Method:  method,
        Params:  paramsBytes,
    }
    return json.Marshal(msg)
}

// Helper: Parse JSON-RPC response
func ParseResponse(data []byte) (*LSPMessage, error) {
    var msg LSPMessage
    if err := json.Unmarshal(data, &msg); err != nil {
        return nil, err
    }
    if msg.JSONRPC != "2.0" {
        return nil, fmt.Errorf("invalid jsonrpc version: %s",
        msg.JSONRPC)
    }
    return &msg, nil
}
```

```go
// Helper: File path to URI
func FileToURI(path string) string {
    return "file://" + path
}

// Helper: URI to file path
func URIToFile(uri string) string {
    if len(uri) > 7 && uri[:7] == "file://" {
        return uri[7:]
    }
    return uri
}
```

NEW FILE: internal/tools/lsp_client.go

```go
package tools

import (
    "bufio"
    "context"
    "encoding/json"
    "fmt"
    "io"
    "log"
    "os/exec"
    "strconv"
    "strings"
    "sync"
    "time"
)

// LSPClient manages a single LSP server process connection
type LSPClient struct {
    langID       string
    cmd          *exec.Cmd
    stdin        io.WriteCloser
    stdout       io.ReadCloser
    stderr       io.ReadCloser
    reader       *bufio.Reader
    nextID       int
    pending      map[int]chan *LSPMessage
    pendingMu    sync.Mutex
    capabilities ServerCapabilities
    isReady      bool
    readyCh      chan struct{}
    diagnostics  map[string]*PublishDiagnosticsParams // uri ->
        diagnostics
```

```
    diagMu       sync.RWMutex
    stopCh       chan struct{}
    wg           sync.WaitGroup
}

// NewLSPClient creates a client for a language server
func NewLSPClient(langID, command string, args []string)
        (*LSPClient, error) {
    cmd := exec.Command(command, args...)
    stdin, err := cmd.StdinPipe()
    if err != nil {
        return nil, fmt.Errorf("stdin pipe: %w", err)
    }
    stdout, err := cmd.StdoutPipe()
    if err != nil {
        return nil, fmt.Errorf("stdout pipe: %w", err)
    }
    stderr, err := cmd.StderrPipe()
    if err != nil {
        return nil, fmt.Errorf("stderr pipe: %w", err)
    }

    if err := cmd.Start(); err != nil {
        return nil, fmt.Errorf("start lsp server: %w", err)
    }

    client := &LSPClient{
        langID:      langID,
        cmd:         cmd,
        stdin:       stdin,
        stdout:      stdout,
        stderr:      stderr,
        reader:      bufio.NewReader(stdout),
        nextID:      1,
        pending:     make(map[int]chan *LSPMessage),
        readyCh:     make(chan struct{}),
        diagnostics: make(map[string]*PublishDiagnosticsParams),
        stopCh:      make(chan struct{}),
    }

    // Start reading responses
    client.wg.Add(1)
    go client.readLoop()

    // Start stderr logger
    client.wg.Add(1)
    go client.stderrLoop()
```

```
    return client, nil
}

// Initialize sends the LSP initialize request
func (c *LSPClient) Initialize(ctx context.Context, rootPath
        string) (*InitializeResult, error) {
    params := InitializeParams{
        ProcessID: os.Getpid(),
        RootURI:   FileToURI(rootPath),
        Capabilities: ClientCapabilities{
            TextDocument: TextDocumentClientCapabilities{
                PublishDiagnostics:
        PublishDiagnosticsCapabilities{
                    RelatedInformation: true,
                    VersionSupport:     true,
                },
            },
        },
    }

    id := c.nextID()
    _, data, err := NewRequest(id, "initialize", params)
    if err != nil {
        return nil, err
    }

    resp, err := c.sendAndWait(ctx, id, data)
    if err != nil {
        return nil, err
    }

    if resp.Error != nil {
        return nil, fmt.Errorf("initialize error: %s (code=%d)",
        resp.Error.Message, resp.Error.Code)
    }

    var result InitializeResult
    if err := json.Unmarshal(resp.Result, &result); err != nil {
        return nil, err
    }

    c.capabilities = result.Capabilities
    c.isReady = true
    close(c.readyCh)

    // Send initialized notification
    notif, _ := NewNotification("initialized",
        map[string]interface{}{})
```

```
    c.sendRaw(notif)

    return &result, nil
}

// DidOpen notifies the server that a file is open
func (c *LSPClient) DidOpen(uri, languageID string, version int,
        content string) error {
    params := DidOpenTextDocumentParams{
        TextDocument: TextDocumentItem{
            URI:        uri,
            LanguageID: languageID,
            Version:    version,
            Text:       content,
        },
    }
    data, err := NewNotification("textDocument/didOpen", params)
    if err != nil {
        return err
    }
    return c.sendRaw(data)
}

// DidChange notifies the server of file changes
func (c *LSPClient) DidChange(uri string, version int,
        newContent string) error {
    params := DidChangeTextDocumentParams{
        TextDocument: VersionedTextDocumentIdentifier{
            URI:     uri,
            Version: version,
        },
        ContentChanges: []TextDocumentContentChangeEvent{
            {Text: newContent},
        },
    }
    data, err := NewNotification("textDocument/didChange",
        params)
    if err != nil {
        return err
    }
    return c.sendRaw(data)
}

// GetDiagnostics returns the latest diagnostics for a file
func (c *LSPClient) GetDiagnostics(uri string) ([]Diagnostic,
        bool) {
    c.diagMu.RLock()
    defer c.diagMu.RUnlock()
```

```
    if diag, ok := c.diagnostics[uri]; ok {
        return diag.Diagnostics, true
    }
    return nil, false
}

// GetAllDiagnostics returns all known diagnostics
func (c *LSPClient) GetAllDiagnostics() map[string][]Diagnostic {
    c.diagMu.RLock()
    defer c.diagMu.RUnlock()
    result := make(map[string][]Diagnostic)
    for uri, diag := range c.diagnostics {
        result[uri] = diag.Diagnostics
    }
    return result
}

// PullDiagnostics requests diagnostics via the pull model (LSP
        3.17+)
func (c *LSPClient) PullDiagnostics(ctx context.Context, uri
        string) (*DocumentDiagnosticReport, error) {
    if !c.capabilities.DiagnosticProvider.WorkspaceDiagnostics {
        return nil, fmt.Errorf("server does not support pull
        diagnostics")
    }

    params := DocumentDiagnosticParams{
        TextDocument: TextDocumentIdentifier{URI: uri},
    }

    id := c.nextID()
    _, data, err := NewRequest(id, "textDocument/diagnostic",
        params)
    if err != nil {
        return nil, err
    }

    resp, err := c.sendAndWait(ctx, id, data)
    if err != nil {
        return nil, err
    }

    if resp.Error != nil {
        return nil, fmt.Errorf("diagnostics error: %s",
        resp.Error.Message)
    }

    var report DocumentDiagnosticReport
```

```
    if err := json.Unmarshal(resp.Result, &report); err != nil {
        return nil, err
    }
    return &report, nil
}

// Shutdown gracefully stops the LSP server
func (c *LSPClient) Shutdown(ctx context.Context) error {
    // Send shutdown request
    id := c.nextID()
    _, data, _ := NewRequest(id, "shutdown", nil)
    _, _ = c.sendAndWait(ctx, id, data)

    // Send exit notification
    exitNotif, _ := NewNotification("exit", nil)
    c.sendRaw(exitNotif)

    close(c.stopCh)
    c.wg.Wait()

    if c.stdin != nil {
        c.stdin.Close()
    }
    if c.cmd.Process != nil {
        c.cmd.Process.Kill()
    }
    return nil
}

func (c *LSPClient) nextID() int {
    c.pendingMu.Lock()
    defer c.pendingMu.Unlock()
    id := c.nextID
    c.nextID++
    return id
}

func (c *LSPClient) sendRaw(data []byte) error {
    header := fmt.Sprintf("Content-Length: %d\r\n\r\n",
        len(data))
    if _, err := c.stdin.Write([]byte(header)); err != nil {
        return err
    }
    _, err := c.stdin.Write(data)
    return err
}
```

```
func (c *LSPClient) sendAndWait(ctx context.Context, id int,
        data []byte) (*LSPMessage, error) {
    ch := make(chan *LSPMessage, 1)
    c.pendingMu.Lock()
    c.pending[id] = ch
    c.pendingMu.Unlock()

    defer func() {
        c.pendingMu.Lock()
        delete(c.pending, id)
        c.pendingMu.Unlock()
    }()

    if err := c.sendRaw(data); err != nil {
        return nil, err
    }

    select {
    case <-ctx.Done():
        return nil, ctx.Err()
    case resp := <-ch:
        return resp, nil
    case <-time.After(30 * time.Second):
        return nil, fmt.Errorf("LSP request timeout")
    }
}

func (c *LSPClient) readLoop() {
    defer c.wg.Done()
    for {
        select {
        case <-c.stopCh:
            return
        default:
        }

        // Read Content-Length header
        line, err := c.reader.ReadString('\n')
        if err != nil {
            if err != io.EOF {
                log.Printf("[LSPClient %s] read error: %v",
        c.langID, err)
            }
            return
        }

        if !strings.HasPrefix(line, "Content-Length: ") {
            continue
```

```
}

lengthStr := strings.TrimSpace(strings.TrimPrefix(line,
"Content-Length: "))
length, err := strconv.Atoi(lengthStr)
if err != nil {
    continue
}

// Read blank line
_, _ = c.reader.ReadString('\n')

// Read body
body := make([]byte, length)
_, err = io.ReadFull(c.reader, body)
if err != nil {
    log.Printf("[LSPClient %s] body read error: %v",
c.langID, err)
    return
}

// Parse message
msg, err := ParseResponse(body)
if err != nil {
    log.Printf("[LSPClient %s] parse error: %v",
c.langID, err)
    continue
}

// Handle notifications
if msg.Method != "" && msg.ID == nil {
    c.handleNotification(msg)
    continue
}

// Handle responses
if msg.ID != nil {
    var id int
    switch v := msg.ID.(type) {
    case float64:
        id = int(v)
    case int:
        id = v
    case string:
        id, _ = strconv.Atoi(v)
    }

    c.pendingMu.Lock()
```

```
            ch, ok := c.pending[id]
            c.pendingMu.Unlock()
            if ok {
                ch <- msg
            }
        }
    }
}

func (c *LSPClient) handleNotification(msg *LSPMessage) {
    switch msg.Method {
    case "textDocument/publishDiagnostics":
        var params PublishDiagnosticsParams
        if err := json.Unmarshal(msg.Params, &params); err !=
        nil {
            log.Printf("[LSPClient %s] diagnostics parse error:
        %v", c.langID, err)
            return
        }
        c.diagMu.Lock()
        c.diagnostics[params.URI] = &params
        c.diagMu.Unlock()
        log.Printf("[LSPClient %s] received %d diagnostics for
        %s",
            c.langID, len(params.Diagnostics),
        URIToFile(params.URI))
    default:
        log.Printf("[LSPClient %s] unhandled notification: %s",
        c.langID, msg.Method)
    }
}

func (c *LSPClient) stderrLoop() {
    defer c.wg.Done()
    scanner := bufio.NewScanner(c.stderr)
    for scanner.Scan() {
        select {
        case <-c.stopCh:
            return
        default:
            log.Printf("[LSPClient %s stderr] %s", c.langID,
        scanner.Text())
        }
    }
}
```

NEW FILE: internal/tools/lsp_server_manager.go

```go
package tools

import (
    "context"
    "fmt"
    "log"
    "os/exec"
    "path/filepath"
    "strings"
    "sync"
)

// LSPServerConfig defines how to launch a language server
type LSPServerConfig struct {
    Disabled bool     `json:"disabled" yaml:"disabled"`
    Command  string   `json:"command" yaml:"command"`
    Args     []string `json:"args" yaml:"args"`
}

// LSPServerManager manages multiple LSP clients keyed by
    language
type LSPServerManager struct {
    clients map[string]*LSPClient // langID -> client
    configs map[string]LSPServerConfig
    mu      sync.RWMutex
}

// DefaultLSPConfigs maps file extensions / language IDs to
    server commands
var DefaultLSPConfigs = map[string]LSPServerConfig{
    "go":         {Command: "gopls", Args: []string{"serve"}},
    "typescript": {Command: "typescript-language-server", Args:
        []string{"--stdio"}},
    "javascript": {Command: "typescript-language-server", Args:
        []string{"--stdio"}},
    "python":     {Command: "pyright-langserver", Args:
        []string{"--stdio"}},
    "rust":       {Command: "rust-analyzer", Args: []string{}},
    "swift":      {Command: "sourcekit-lsp", Args: []string{}},
    "terraform":  {Command: "terraform-ls", Args:
        []string{"serve"}},
    "java":       {Command: "jdtls", Args: []string{}},
    "cpp":        {Command: "clangd", Args: []string{}},
    "c":          {Command: "clangd", Args: []string{}},
    "ruby":       {Command: "ruby-lsp", Args: []string{}},
```

```
    "php":        {Command: "intelephense", Args: []string{"--
        stdio"}},
    "html":       {Command: "vscode-html-language-server", Args:
        []string{"--stdio"}},
    "css":        {Command: "vscode-css-language-server", Args:
        []string{"--stdio"}},
    "json":       {Command: "vscode-json-language-server", Args:
        []string{"--stdio"}},
    "yaml":       {Command: "yaml-language-server", Args:
        []string{"--stdio"}},
    "dockerfile": {Command: "docker-langserver", Args:
        []string{"--stdio"}},
    "bash":       {Command: "bash-language-server", Args:
        []string{"start"}},
    "lua":        {Command: "lua-language-server", Args:
        []string{}},
}

func NewLSPServerManager(configs map[string]LSPServerConfig)
        *LSPServerManager {
    if configs == nil {
        configs = make(map[string]LSPServerConfig)
    }
    return &LSPServerManager{
        clients: make(map[string]*LSPClient),
        configs: configs,
    }
}

// StartLanguageServer starts an LSP client for a language if not
        already running
func (sm *LSPServerManager) StartLanguageServer(
    ctx context.Context,
    langID string,
    rootPath string,
) (*LSPClient, error) {
    sm.mu.Lock()
    defer sm.mu.Unlock()

    if client, ok := sm.clients[langID]; ok {
        return client, nil
    }

    cfg, ok := sm.configs[langID]
    if !ok {
        cfg, ok = DefaultLSPConfigs[langID]
    }
    if !ok || cfg.Disabled {
```

```
            return nil, fmt.Errorf("no LSP config for language: %s",
                langID)
        }

        // Verify command exists
        if _, err := exec.LookPath(cfg.Command); err != nil {
            return nil, fmt.Errorf("LSP command not found: %s",
                cfg.Command)
        }

        client, err := NewLSPClient(langID, cfg.Command, cfg.Args)
        if err != nil {
            return nil, err
        }

        // Initialize with root path
        if _, err := client.Initialize(ctx, rootPath); err != nil {
            client.Shutdown(ctx)
            return nil, fmt.Errorf("initialize failed: %w", err)
        }

        sm.clients[langID] = client
        log.Printf("[LSPServerManager] Started %s LSP for %s",
            langID, rootPath)
        return client, nil
    }

    // GetClient returns the client for a language
    func (sm *LSPServerManager) GetClient(langID string)
            (*LSPClient, bool) {
        sm.mu.RLock()
        defer sm.mu.RUnlock()
        c, ok := sm.clients[langID]
        return c, ok
    }

    // DetectLanguage detects language ID from file extension
    func DetectLanguage(filePath string) string {
        ext := strings.ToLower(filepath.Ext(filePath))
        switch ext {
        case ".go":
            return "go"
        case ".ts", ".tsx":
            return "typescript"
        case ".js", ".jsx", ".mjs":
            return "javascript"
        case ".py", ".pyw":
            return "python"
```

```
    case ".rs":
        return "rust"
    case ".swift":
        return "swift"
    case ".tf", ".tfvars":
        return "terraform"
    case ".java":
        return "java"
    case ".cpp", ".cc", ".cxx":
        return "cpp"
    case ".c", ".h":
        return "c"
    case ".rb":
        return "ruby"
    case ".php":
        return "php"
    case ".html", ".htm":
        return "html"
    case ".css", ".scss", ".sass":
        return "css"
    case ".json":
        return "json"
    case ".yaml", ".yml":
        return "yaml"
    case ".dockerfile", "Dockerfile":
        return "dockerfile"
    case ".sh", ".bash":
        return "bash"
    case ".lua":
        return "lua"
    default:
        return ""
    }
}

// TouchFile opens/changes a file in the LSP server to trigger
        diagnostics
func (sm *LSPServerManager) TouchFile(ctx context.Context,
        filePath string, readContent bool) error {
    langID := DetectLanguage(filePath)
    if langID == "" {
        return fmt.Errorf("cannot detect language for %s",
        filePath)
    }

    client, err := sm.StartLanguageServer(ctx, langID,
        filepath.Dir(filePath))
    if err != nil {
```

```
            return err
        }

        uri := FileToURI(filePath)

        if readContent {
            content, err := os.ReadFile(filePath)
            if err != nil {
                return err
            }
            client.DidOpen(uri, langID, 1, string(content))
        } else {
            // Just trigger with empty open then change
            client.DidOpen(uri, langID, 1, "")
        }

        return nil
    }

    // GetDiagnosticsForFile returns diagnostics for a specific file
    func (sm *LSPServerManager) GetDiagnosticsForFile(filePath
            string) ([]Diagnostic, error) {
        langID := DetectLanguage(filePath)
        if langID == "" {
            return nil, fmt.Errorf("cannot detect language for %s",
            filePath)
        }

        sm.mu.RLock()
        client, ok := sm.clients[langID]
        sm.mu.RUnlock()
        if !ok {
            return nil, fmt.Errorf("no LSP client for language: %s",
            langID)
        }

        uri := FileToURI(filePath)
        diags, ok := client.GetDiagnostics(uri)
        if !ok {
            return nil, nil
        }
        return diags, nil
    }

    // ShutdownAll gracefully stops all language servers
    func (sm *LSPServerManager) ShutdownAll(ctx context.Context) {
        sm.mu.Lock()
        defer sm.mu.Unlock()
```

```
    for langID, client := range sm.clients {
        if err := client.Shutdown(ctx); err != nil {
            log.Printf("[LSPServerManager] Error shutting down
        %s: %v", langID, err)
        }
        delete(sm.clients, langID)
    }
}
```

NEW FILE: internal/tools/diagnostics_tool.go

```
package tools

import (
    "context"
    "encoding/json"
    "fmt"
    "strings"
)

// DiagnosticsTool implements the `diagnostics` tool exposed to
        the AI agent
type DiagnosticsTool struct {
    lspManager *LSPServerManager
}

func NewDiagnosticsTool(manager *LSPServerManager)
        *DiagnosticsTool {
    return &DiagnosticsTool{lspManager: manager}
}

func (dt *DiagnosticsTool) Name() string { return "diagnostics" }

func (dt *DiagnosticsTool) Description() string {
    return "Get LSP diagnostics for the current file or entire
        project. " +
        "Use this after editing code to verify correctness."
}

type diagnosticsInput struct {
    FilePath string `json:"file_path,omitempty"`
}

type diagnosticsOutput struct {
    FilePath   string       `json:"file_path"`
    ErrorCount int          `json:"error_count"`
    WarningCount int        `json:"warning_count"`
    Diagnostics []DiagnosticSummary `json:"diagnostics"`
```

```
}

type DiagnosticSummary struct {
    Line     int    `json:"line"`
    Column   int    `json:"column"`
    Severity string `json:"severity"`
    Message  string `json:"message"`
    Source   string `json:"source,omitempty"`
    Code     string `json:"code,omitempty"`
}

func (dt *DiagnosticsTool) Execute(ctx context.Context, args
        json.RawMessage) (string, error) {
    var input diagnosticsInput
    if err := json.Unmarshal(args, &input); err != nil {
        return "", fmt.Errorf("invalid diagnostics input: %w",
        err)
    }

    var allDiags map[string][]Diagnostic
    var err error

    if input.FilePath != "" {
        // Touch file to ensure LSP has processed it
        _ = dt.lspManager.TouchFile(ctx, input.FilePath, true)
        diags, derr :=
        dt.lspManager.GetDiagnosticsForFile(input.FilePath)
        if derr != nil {
            return "", derr
        }
        allDiags = map[string][]Diagnostic{input.FilePath: diags}
    } else {
        // Get diagnostics from all active LSP clients
        allDiags = make(map[string][]Diagnostic)
        for _, langID := range []string{"go", "typescript",
        "python", "rust"} {
            if client, ok := dt.lspManager.GetClient(langID); ok
        {
                for uri, diags := range
        client.GetAllDiagnostics() {
                    allDiags[URIToFile(uri)] = diags
                }
            }
        }
    }

    // Format output
    var outputs []diagnosticsOutput
```

```
    for filePath, diags := range allDiags {
        out := diagnosticsOutput{FilePath: filePath}
        for _, d := range diags {
            switch d.Severity {
            case SeverityError:
                out.ErrorCount++
            case SeverityWarning:
                out.WarningCount++
            }
            out.Diagnostics = append(out.Diagnostics,
        DiagnosticSummary{
                Line:     d.Range.Start.Line,
                Column:   d.Range.Start.Character,
                Severity: severityToString(d.Severity),
                Message:  d.Message,
                Source:   d.Source,
                Code:     fmt.Sprintf("%v", d.Code),
            })
        }
        outputs = append(outputs, out)
    }

    result, _ := json.MarshalIndent(outputs, "", "  ")
    return string(result), nil
}

func severityToString(s DiagnosticSeverity) string {
    switch s {
    case SeverityError:
        return "error"
    case SeverityWarning:
        return "warning"
    case SeverityInformation:
        return "info"
    case SeverityHint:
        return "hint"
    default:
        return "unknown"
    }
}

// AfterEditHook should be called after any file edit to get
        fresh diagnostics
func (dt *DiagnosticsTool) AfterEditHook(ctx context.Context,
        filePath string) ([]Diagnostic, error) {
    _ = dt.lspManager.TouchFile(ctx, filePath, true)

    // Poll briefly for diagnostics to arrive
```

```
    for i := 0; i < 10; i++ {
        diags, err :=
        dt.lspManager.GetDiagnosticsForFile(filePath)
        if err == nil && len(diags) > 0 {
            return diags, nil
        }
        select {
        case <-ctx.Done():
            return nil, ctx.Err()
        default:
            // 100ms * 10 = 1s max wait
        }
    }

    return dt.lspManager.GetDiagnosticsForFile(filePath)
}
```

Integration Point in HelixCode's Architecture

- The `DiagnosticsTool` is registered in the tool registry (`internal/tools/registry.go`)
- After every `EditTool` or `WriteTool` execution, the session calls `DiagnosticsTool.AfterEditHook()`
- Results are appended to the next LLM prompt via `internal/llm/prompt_builder.go`
- The `LSPServerManager` is instantiated once per session and stored in `Session.LSPManager`

Anti-Bluff Test

```
# TEST 1: Verify LSP JSON-RPC framing
go test ./internal/tools/ -run TestLSPProtocol_Framing -v

# TEST 2: Start gopls, open a Go file, verify initialize
        handshake
go test ./internal/tools/ -run TestLSPClient_Initialize -v

# TEST 3: Edit a Go file with a syntax error, verify diagnostics
        published
go test ./internal/tools/ -run TestLSPClient_Diagnostics -v

# TEST 4: End-to-end: edit tool runs, then diagnostics appear in
        next prompt
go test ./internal/session/ -run TestDiagnosticsInPrompt -v

# TEST 5: Server manager starts correct server per language
go test ./internal/tools/ -run
        TestLSPServerManager_DetectLanguage -v
```

Integration Verification

- [ ] `gopls` or `typescript-language-server` starts successfully
- [ ] `textDocument/publishDiagnostics` notifications are parsed correctly
- [ ] Diagnostics appear in AI context after file edits
- [ ] LSP server processes shut down cleanly on session end
- [ ] 15+ language servers mapped in `DefaultLSPConfigs`

Feature 3: Plugin System with 25+ Lifecycle Hooks

Source Location (in original agent)

- `opencode/src/plugins/plugin-manager.ts` — Plugin loading and lifecycle
- `opencode/src/plugins/hook-registry.ts` — Hook registration/dispatch
- `opencode/src/plugins/npm-loader.ts` — npm package loading
- `opencode/src/plugins/builtin.ts` — Built-in plugin list

Target Location (in HelixCode)

- `internal/plugins/` (NEW directory)
  - `internal/plugins/manager.go`
  - `internal/plugins/hook.go`
  - `internal/plugins/loader.go`
  - `internal/plugins/npm.go`
  - `internal/plugins/plugin.go`
  - `internal/plugins/builtins.go`
  - `internal/plugins/config.go`
- `internal/session/session.go` (MODIFY)
- `cmd/cli/main.go` (MODIFY)
- `internal/tools/registry.go` (MODIFY)

Exact Code Changes

NEW FILE: `internal/plugins/hook.go`

```
package plugins

import (
    "context"
    "fmt"
    "sync"
)
```

```go
// HookPoint defines the 25+ lifecycle hooks
type HookPoint string

const (
    // Init hooks
    HookPreInit    HookPoint = "pre-init"
    HookPostInit   HookPoint = "post-init"

    // Session hooks
    HookPreSessionStart  HookPoint = "pre-session-start"
    HookPostSessionStart HookPoint = "post-session-start"
    HookPreSessionEnd    HookPoint = "pre-session-end"
    HookPostSessionEnd   HookPoint = "post-session-end"

    // Tool hooks
    HookPreToolExecute  HookPoint = "pre-tool-execute"
    HookPostToolExecute HookPoint = "post-tool-execute"
    HookToolError       HookPoint = "tool-error"

    // File hooks
    HookPreFileRead   HookPoint = "pre-file-read"
    HookPostFileRead  HookPoint = "post-file-read"
    HookPreFileWrite  HookPoint = "pre-file-write"
    HookPostFileWrite HookPoint = "post-file-write"
    HookPreFileEdit   HookPoint = "pre-file-edit"
    HookPostFileEdit  HookPoint = "post-file-edit"

    // LLM hooks
    HookPreLLMCall    HookPoint = "pre-llm-call"
    HookPostLLMCall   HookPoint = "post-llm-call"
    HookPreStream     HookPoint = "pre-stream"
    HookOnStreamChunk HookPoint = "on-stream-chunk"
    HookPostStream    HookPoint = "post-stream"

    // Context hooks
    HookPreContextBuild  HookPoint = "pre-context-build"
    HookPostContextBuild HookPoint = "post-context-build"
    HookPreContextTrim   HookPoint = "pre-context-trim"

    // Prompt hooks
    HookPreSystemPrompt  HookPoint = "pre-system-prompt"
    HookPostSystemPrompt HookPoint = "post-system-prompt"
    HookPreUserPrompt    HookPoint = "pre-user-prompt"
    HookPostUserPrompt   HookPoint = "post-user-prompt"

    // Command hooks
    HookPreCustomCommand  HookPoint = "pre-custom-command"
    HookPostCustomCommand HookPoint = "post-custom-command"
```

```
    // Lifecycle
    HookShutdown HookPoint = "shutdown"
)

// AllHookPoints is the complete list for validation
var AllHookPoints = []HookPoint{
    HookPreInit, HookPostInit,
    HookPreSessionStart, HookPostSessionStart,
        HookPreSessionEnd, HookPostSessionEnd,
    HookPreToolExecute, HookPostToolExecute, HookToolError,
    HookPreFileRead, HookPostFileRead, HookPreFileWrite,
        HookPostFileWrite,
    HookPreFileEdit, HookPostFileEdit,
    HookPreLLMCall, HookPostLLMCall, HookPreStream,
        HookOnStreamChunk, HookPostStream,
    HookPreContextBuild, HookPostContextBuild,
        HookPreContextTrim,
    HookPreSystemPrompt, HookPostSystemPrompt,
        HookPreUserPrompt, HookPostUserPrompt,
    HookPreCustomCommand, HookPostCustomCommand,
    HookShutdown,
}

// HookContext carries data to hook handlers
type HookContext struct {
    Context    context.Context
    SessionID  string
    Data       map[string]interface{}
    Cancel     bool       // Set to true by a hook to cancel the
        operation
    Result     interface{} // Hooks can modify results
    Error      error
}

func NewHookContext(ctx context.Context, sessionID string)
        *HookContext {
    return &HookContext{
        Context:   ctx,
        SessionID: sessionID,
        Data:      make(map[string]interface{}),
    }
}

// HookHandler is a function that handles a hook
type HookHandler func(*HookContext) error

// HookRegistry stores and dispatches hooks
type HookRegistry struct {
```

```
    hooks map[HookPoint][]HookHandler
    mu    sync.RWMutex
}

func NewHookRegistry() *HookRegistry {
    return &HookRegistry{
        hooks: make(map[HookPoint][]HookHandler),
    }
}

// Register adds a handler for a hook point
func (hr *HookRegistry) Register(point HookPoint, handler
        HookHandler) error {
    valid := false
    for _, h := range AllHookPoints {
        if h == point {
            valid = true
            break
        }
    }
    if !valid {
        return fmt.Errorf("invalid hook point: %s", point)
    }

    hr.mu.Lock()
    defer hr.mu.Unlock()
    hr.hooks[point] = append(hr.hooks[point], handler)
    return nil
}

// UnregisterAll removes all handlers for a hook point
func (hr *HookRegistry) UnregisterAll(point HookPoint) {
    hr.mu.Lock()
    defer hr.mu.Unlock()
    delete(hr.hooks, point)
}

// Dispatch calls all handlers for a hook point synchronously
func (hr *HookRegistry) Dispatch(point HookPoint, ctx
        *HookContext) error {
    hr.mu.RLock()
    handlers := hr.hooks[point]
    hr.mu.RUnlock()

    for _, h := range handlers {
        if err := h(ctx); err != nil {
            ctx.Error = err
            return err
```

```
        }
        if ctx.Cancel {
            return fmt.Errorf("hook %s cancelled the operation",
        point)
        }
    }
    return nil
}

// DispatchAsync calls all handlers asynchronously (fire-and-
        forget)
func (hr *HookRegistry) DispatchAsync(point HookPoint, ctx
        *HookContext) {
    go func() {
        _ = hr.Dispatch(point, ctx)
    }()
}

// HasHandlers returns true if any handler is registered
func (hr *HookRegistry) HasHandlers(point HookPoint) bool {
    hr.mu.RLock()
    defer hr.mu.RUnlock()
    return len(hr.hooks[point]) > 0
}
```

NEW FILE: internal/plugins/plugin.go

```
package plugins

import (
    "context"
    "fmt"
    "time"
)

// Plugin is the interface all plugins must implement
type Plugin interface {
    // Metadata
    Name() string
    Version() string
    Description() string
    Author() string

    // Lifecycle
    Init(ctx context.Context, config map[string]interface{})
        error
    Shutdown(ctx context.Context) error
```

```
    // Hooks returns the list of hooks this plugin wants to
        register
    Hooks() map[HookPoint]HookHandler
}

// PluginDescriptor is the manifest for a plugin
type PluginDescriptor struct {
    Name        string                 `json:"name" yaml:"name"`
    Version     string                 `json:"version"
        yaml:"version"`
    Description string                 `json:"description"
        yaml:"description"`
    Author      string                 `json:"author"
        yaml:"author"`
    Main        string                 `json:"main"
        yaml:"main"`          // Entry point
    Hooks       []string               `json:"hooks"
        yaml:"hooks"`        // Hook points
    Config      map[string]interface{} `json:"config"
        yaml:"config"`     // Default config
    Type        PluginType             `json:"type" yaml:"type"`
    Source      string                 `json:"source"
        yaml:"source"`     // npm package name, git URL, or local
        path
}

type PluginType string

const (
    PluginTypeGo        PluginType = "go"
    PluginTypeNative    PluginType = "native"
    PluginTypeNPM       PluginType = "npm"
    PluginTypeWASM      PluginType = "wasm"
    PluginTypeBuiltIn   PluginType = "builtin"
)

// BasePlugin provides default implementations
type BasePlugin struct {
    PluginDescriptor
}

func (p *BasePlugin) Name() string        { return
        p.PluginDescriptor.Name }
func (p *BasePlugin) Version() string     { return
        p.PluginDescriptor.Version }
func (p *BasePlugin) Description() string  { return
        p.PluginDescriptor.Description }
```

```
func (p *BasePlugin) Author() string      { return
        p.PluginDescriptor.Author }

func (p *BasePlugin) Init(ctx context.Context, config
        map[string]interface{}) error {
    return nil
}

func (p *BasePlugin) Shutdown(ctx context.Context) error {
    return nil
}

func (p *BasePlugin) Hooks() map[HookPoint]HookHandler {
    return nil
}

// PluginState tracks runtime state
type PluginState struct {
    Descriptor PluginDescriptor
    Instance   Plugin
    LoadedAt   time.Time
    LastError  error
    Active     bool
}
```

NEW FILE: internal/plugins/loader.go

```
package plugins

import (
    "context"
    "encoding/json"
    "fmt"
    "log"
    "os"
    "path/filepath"
    "plugin"
    "strings"
)

// PluginLoader discovers and loads plugins from filesystem
type PluginLoader struct {
    searchPaths []string
}

func NewPluginLoader(searchPaths []string) *PluginLoader {
    if len(searchPaths) == 0 {
        searchPaths = []string{
```

```
            os.ExpandEnv("$HOME/.helix/plugins"),
            "./plugins",
        }
    }
    return &PluginLoader{searchPaths: searchPaths}
}

// Discover finds all plugin manifests in search paths
func (pl *PluginLoader) Discover() ([]PluginDescriptor, error) {
    var descriptors []PluginDescriptor

    for _, path := range pl.searchPaths {
        entries, err := os.ReadDir(path)
        if err != nil {
            if os.IsNotExist(err) {
                continue
            }
            return nil, err
        }

        for _, entry := range entries {
            if entry.IsDir() {
                desc, err := pl.loadManifest(filepath.Join(path,
        entry.Name()))
                if err != nil {
                    log.Printf("[PluginLoader] Failed to load
        manifest from %s: %v", entry.Name(), err)
                    continue
                }
                descriptors = append(descriptors, desc)
            }
        }
    }

    return descriptors, nil
}

func (pl *PluginLoader) loadManifest(pluginDir string)
        (PluginDescriptor, error) {
    manifestPath := filepath.Join(pluginDir, "plugin.json")
    data, err := os.ReadFile(manifestPath)
    if err != nil {
        // Try YAML fallback
        manifestPath = filepath.Join(pluginDir, "plugin.yaml")
        data, err = os.ReadFile(manifestPath)
        if err != nil {
            return PluginDescriptor{}, fmt.Errorf("no manifest
        found: %w", err)
```

```
        }
        // YAML parsing would use go-yaml here
        return PluginDescriptor{},
        fmt.Errorf("YAML manifests not yet implemented")
    }

    var desc PluginDescriptor
    if err := json.Unmarshal(data, &desc); err != nil {
        return PluginDescriptor{}, err
    }

    // Resolve relative paths
    if !filepath.IsAbs(desc.Main) {
        desc.Main = filepath.Join(pluginDir, desc.Main)
    }

    return desc, nil
}

// LoadGoPlugin loads a Go plugin (.so)
func (pl *PluginLoader) LoadGoPlugin(desc PluginDescriptor)
        (Plugin, error) {
    if desc.Type != PluginTypeGo && desc.Type !=
        PluginTypeNative {
        return nil, fmt.Errorf("not a Go plugin: %s", desc.Type)
    }

    soPath := desc.Main
    if !strings.HasSuffix(soPath, ".so") {
        soPath = soPath + ".so"
    }

    p, err := plugin.Open(soPath)
    if err != nil {
        return nil, fmt.Errorf("plugin.Open: %w", err)
    }

    // Look up exported symbol "Plugin"
    sym, err := p.Lookup("Plugin")
    if err != nil {
        return nil, fmt.Errorf("plugin.Lookup(Plugin): %w", err)
    }

    plug, ok := sym.(Plugin)
    if !ok {
        return nil, fmt.Errorf("symbol Plugin does not implement
        plugins.Plugin interface")
    }
```

```
    return plug, nil
}
```

NEW FILE: internal/plugins/npm.go

```
package plugins

import (
    "context"
    "encoding/json"
    "fmt"
    "log"
    "os"
    "os/exec"
    "path/filepath"
)

// NPMPluginLoader loads TypeScript/JavaScript plugins via npm
type NPMPluginLoader struct {
    nodeModulesPath string
    nodeBinary      string
}

func NewNPMPluginLoader() *NPMPluginLoader {
    return &NPMPluginLoader{
        nodeModulesPath: os.ExpandEnv("$HOME/.helix/
        node_modules"),
        nodeBinary:      "node",
    }
}

// InstallPackage runs npm install for a plugin package
func (nl *NPMPluginLoader) InstallPackage(ctx context.Context,
        packageName string) error {
    log.Printf("[NPMPluginLoader] Installing %s...", packageName)

    cmd := exec.CommandContext(ctx, "npm", "install", "--
        prefix", nl.nodeModulesPath, packageName)
    output, err := cmd.CombinedOutput()
    if err != nil {
        return fmt.Errorf("npm install failed: %w\n%s", err,
        string(output))
    }
    return nil
}

// LoadNPMPlugin loads a plugin from an npm package
```

```
func (nl *NPMPluginLoader) LoadNPMPlugin(desc PluginDescriptor)
        (*NPMPluginBridge, error) {
    if desc.Type != PluginTypeNPM {
        return nil, fmt.Errorf("not an npm plugin: %s",
        desc.Type)
    }

    pkgDir := filepath.Join(nl.nodeModulesPath, "node_modules",
        desc.Source)
    manifestPath := filepath.Join(pkgDir, "helix-plugin.json")

    data, err := os.ReadFile(manifestPath)
    if err != nil {
        return nil, fmt.Errorf("no helix-plugin.json found: %w",
        err)
    }

    var npmManifest struct {
        Main   string            `json:"main"`
        Hooks  map[string]string `json:"hooks"`
        Config map[string]interface{} `json:"config"`
    }
    if err := json.Unmarshal(data, &npmManifest); err != nil {
        return nil, err
    }

    entryPoint := filepath.Join(pkgDir, npmManifest.Main)
    if _, err := os.Stat(entryPoint); err != nil {
        return nil, fmt.Errorf("entry point not found: %s",
        entryPoint)
    }

    return &NPMPluginBridge{
        nodeBinary: nl.nodeBinary,
        entryPoint: entryPoint,
        manifest:   npmManifest,
        descriptor: desc,
    }, nil
}

// NPMPluginBridge bridges Go hooks to a Node.js plugin process
type NPMPluginBridge struct {
    nodeBinary string
    entryPoint string
    manifest   struct {
        Main   string            `json:"main"`
        Hooks  map[string]string `json:"hooks"`
        Config map[string]interface{} `json:"config"`
```

```
    }
    descriptor PluginDescriptor
    cmd        *exec.Cmd
}

func (nb *NPMPluginBridge) Name() string        { return
        nb.descriptor.Name }
func (nb *NPMPluginBridge) Version() string       { return
        nb.descriptor.Version }
func (nb *NPMPluginBridge) Description() string    { return
        nb.descriptor.Description }
func (nb *NPMPluginBridge) Author() string        { return
        nb.descriptor.Author }

func (nb *NPMPluginBridge) Init(ctx context.Context, config
        map[string]interface{}) error {
    // Start the Node.js plugin process as a long-running RPC
        server
    // Communication via stdin/stdout JSON-RPC
    nb.cmd = exec.CommandContext(ctx, nb.nodeBinary,
        nb.entryPoint)
    // Set up pipes and start JSON-RPC bridge
    return nil
}

func (nb *NPMPluginBridge) Shutdown(ctx context.Context) error {
    if nb.cmd != nil && nb.cmd.Process != nil {
        return nb.cmd.Process.Kill()
    }
    return nil
}

func (nb *NPMPluginBridge) Hooks() map[HookPoint]HookHandler {
    result := make(map[HookPoint]HookHandler)
    for hookName, fnName := range nb.manifest.Hooks {
        hookPoint := HookPoint(hookName)
        result[hookPoint] = nb.createBridgeHandler(fnName)
    }
    return result
}

func (nb *NPMPluginBridge) createBridgeHandler(fnName string)
        HookHandler {
    return func(ctx *HookContext) error {
        // Serialize context, send to Node.js process, wait for
        response
        // This is a simplified placeholder
```

```
        log.Printf("[NPMPluginBridge] Calling %s for plugin %s",
        fnName, nb.descriptor.Name)
        return nil
    }
}
```

NEW FILE: internal/plugins/manager.go

```
package plugins

import (
    "context"
    "fmt"
    "log"
    "sync"
    "time"
)

// PluginManager is the central plugin coordinator
type PluginManager struct {
    registry   *HookRegistry
    loader     *PluginLoader
    npmLoader  *NPMPluginLoader
    plugins    map[string]*PluginState
    mu         sync.RWMutex
    searchPaths []string
}

func NewPluginManager(searchPaths []string) *PluginManager {
    return &PluginManager{
        registry:    NewHookRegistry(),
        loader:      NewPluginLoader(searchPaths),
        npmLoader:   NewNPMPluginLoader(),
        plugins:     make(map[string]*PluginState),
        searchPaths: searchPaths,
    }
}

// DiscoverAndLoad finds and loads all available plugins
func (pm *PluginManager) DiscoverAndLoad(ctx context.Context)
        error {
    descriptors, err := pm.loader.Discover()
    if err != nil {
        return err
    }

    for _, desc := range descriptors {
        if err := pm.LoadPlugin(ctx, desc); err != nil {
```

```
            log.Printf("[PluginManager] Failed to load plugin %s:
            %v", desc.Name, err)
            }
        }

        return nil
    }

    // LoadPlugin loads a single plugin by descriptor
    func (pm *PluginManager) LoadPlugin(ctx context.Context, desc
            PluginDescriptor) error {
        pm.mu.Lock()
        defer pm.mu.Unlock()

        if _, exists := pm.plugins[desc.Name]; exists {
            return fmt.Errorf("plugin %s already loaded", desc.Name)
        }

        var plug Plugin
        var err error

        switch desc.Type {
        case PluginTypeGo, PluginTypeNative:
            plug, err = pm.loader.LoadGoPlugin(desc)
        case PluginTypeNPM:
            plug, err = pm.npmLoader.LoadNPMPlugin(desc)
        case PluginTypeBuiltIn:
            plug, err = pm.loadBuiltIn(desc.Name)
        case PluginTypeWASM:
            err = fmt.Errorf("WASM plugins not yet implemented")
        default:
            err = fmt.Errorf("unknown plugin type: %s", desc.Type)
        }

        if err != nil {
            pm.plugins[desc.Name] = &PluginState{
                Descriptor: desc,
                LoadedAt:   time.Now(),
                LastError:  err,
                Active:     false,
            }
            return err
        }

        // Initialize
        if err := plug.Init(ctx, desc.Config); err != nil {
            pm.plugins[desc.Name] = &PluginState{
```

```
            Descriptor: desc,
            Instance:   plug,
            LoadedAt:   time.Now(),
            LastError:  err,
            Active:     false,
        }
        return fmt.Errorf("plugin init failed: %w", err)
    }

    // Register hooks
    if hooks := plug.Hooks(); hooks != nil {
        for point, handler := range hooks {
            if err := pm.registry.Register(point, handler); err !
        = nil {
                log.Printf("[PluginManager] Hook registration
        error for %s: %v", point, err)
            }
        }
    }

    pm.plugins[desc.Name] = &PluginState{
        Descriptor: desc,
        Instance:   plug,
        LoadedAt:   time.Now(),
        Active:     true,
    }

    log.Printf("[PluginManager] Loaded plugin: %s@%s",
        desc.Name, desc.Version)
    return nil
}

// loadBuiltIn loads a built-in plugin by name
func (pm *PluginManager) loadBuiltIn(name string) (Plugin,
        error) {
    if factory, ok := BuiltInPlugins[name]; ok {
        return factory(), nil
    }
    return nil, fmt.Errorf("builtin plugin %s not found", name)
}

// UnloadPlugin shuts down and removes a plugin
func (pm *PluginManager) UnloadPlugin(ctx context.Context, name
        string) error {
    pm.mu.Lock()
    state, ok := pm.plugins[name]
    pm.mu.Unlock()
```

```
    if !ok {
        return fmt.Errorf("plugin %s not found", name)
    }

    if state.Instance != nil {
        // Unregister hooks
        if hooks := state.Instance.Hooks(); hooks != nil {
            for point := range hooks {
                pm.registry.UnregisterAll(point)
            }
        }
        if err := state.Instance.Shutdown(ctx); err != nil {
            log.Printf("[PluginManager] Shutdown error for %s:
        %v", name, err)
        }
    }

    pm.mu.Lock()
    delete(pm.plugins, name)
    pm.mu.Unlock()
    return nil
}

// GetRegistry returns the hook registry for dispatching
func (pm *PluginManager) GetRegistry() *HookRegistry {
    return pm.registry
}

// GetPlugin returns a loaded plugin
func (pm *PluginManager) GetPlugin(name string) (*PluginState,
        bool) {
    pm.mu.RLock()
    defer pm.mu.RUnlock()
    p, ok := pm.plugins[name]
    return p, ok
}

// ListPlugins returns all plugin states
func (pm *PluginManager) ListPlugins() []*PluginState {
    pm.mu.RLock()
    defer pm.mu.RUnlock()
    result := make([]*PluginState, 0, len(pm.plugins))
    for _, p := range pm.plugins {
        result = append(result, p)
    }
    return result
}
```

```go
// Shutdown unloads all plugins
func (pm *PluginManager) Shutdown(ctx context.Context) {
    pm.mu.RLock()
    names := make([]string, 0, len(pm.plugins))
    for name := range pm.plugins {
        names = append(names, name)
    }
    pm.mu.RUnlock()

    for _, name := range names {
        _ = pm.UnloadPlugin(ctx, name)
    }
}
```

NEW FILE: internal/plugins/builtins.go

```go
package plugins

import (
    "context"
    "fmt"
    "log"
    "os"
    "time"
)

// BuiltInPlugins maps names to factory functions
var BuiltInPlugins = map[string]func() Plugin{
    "audit-log":     NewAuditLogPlugin,
    "token-counter": NewTokenCounterPlugin,
    "cost-tracker":  NewCostTrackerPlugin,
    "safety-guard":  NewSafetyGuardPlugin,
}

// AuditLogPlugin logs all tool executions to disk
type AuditLogPlugin struct {
    BasePlugin
    logFile *os.File
}

func NewAuditLogPlugin() Plugin {
    return &AuditLogPlugin{
        BasePlugin: BasePlugin{PluginDescriptor:
        PluginDescriptor{
            Name: "audit-log", Version: "1.0.0",
            Description: "Logs all tool executions for audit",
            Type: PluginTypeBuiltIn,
        }},
```

```
    }
}

func (p *AuditLogPlugin) Init(ctx context.Context, config
        map[string]interface{}) error {
    path := os.ExpandEnv("$HOME/.helix/audit.log")
    if p, ok := config["path"]; ok {
        path = fmt.Sprintf("%v", p)
    }
    f, err := os.OpenFile(path, os.O_APPEND|os.O_CREATE|
        os.O_WRONLY, 0644)
    if err != nil {
        return err
    }
    p.logFile = f
    return nil
}

func (p *AuditLogPlugin) Shutdown(ctx context.Context) error {
    if p.logFile != nil {
        return p.logFile.Close()
    }
    return nil
}

func (p *AuditLogPlugin) Hooks() map[HookPoint]HookHandler {
    return map[HookPoint]HookHandler{
        HookPostToolExecute: func(ctx *HookContext) error {
            entry := fmt.Sprintf("[%s] session=%s tool=%v
        result=%v\n",
                time.Now().Format(time.RFC3339),
                ctx.SessionID,
                ctx.Data["tool"],
                ctx.Data["result"],
            )
            _, err := p.logFile.WriteString(entry)
            return err
        },
    }
}

// TokenCounterPlugin tracks token usage across sessions
type TokenCounterPlugin struct {
    BasePlugin
    totalTokens int
}

func NewTokenCounterPlugin() Plugin {
```

```
    return &TokenCounterPlugin{
        BasePlugin: BasePlugin{PluginDescriptor:
        PluginDescriptor{
            Name: "token-counter", Version: "1.0.0",
            Description: "Tracks cumulative token usage",
            Type: PluginTypeBuiltIn,
        }},
    }
}

func (p *TokenCounterPlugin) Hooks() map[HookPoint]HookHandler {
    return map[HookPoint]HookHandler{
        HookPostLLMCall: func(ctx *HookContext) error {
            if tokens, ok := ctx.Data["output_tokens"]; ok {
                p.totalTokens += tokens.(int)
            }
            return nil
        },
    }
}

// CostTrackerPlugin estimates cumulative API cost
type CostTrackerPlugin struct {
    BasePlugin
    totalCost float64
}

func NewCostTrackerPlugin() Plugin {
    return &CostTrackerPlugin{
        BasePlugin: BasePlugin{PluginDescriptor:
        PluginDescriptor{
            Name: "cost-tracker", Version: "1.0.0",
            Description: "Tracks estimated API spend",
            Type: PluginTypeBuiltIn,
        }},
    }
}

// SafetyGuardPlugin blocks dangerous operations
type SafetyGuardPlugin struct {
    BasePlugin
    blockedPatterns []string
}

func NewSafetyGuardPlugin() Plugin {
    return &SafetyGuardPlugin{
        BasePlugin: BasePlugin{PluginDescriptor:
        PluginDescriptor{
```

```
            Name: "safety-guard", Version: "1.0.0",
            Description: "Prevents execution of dangerous
        commands",
            Type: PluginTypeBuiltIn,
        }},
        blockedPatterns: []string{
            "rm -rf /", "mkfs.", "dd if=/dev/zero",
            "> /etc/passwd", "curl .* | sh",
        },
    }
}

func (p *SafetyGuardPlugin) Hooks() map[HookPoint]HookHandler {
    return map[HookPoint]HookHandler{
        HookPreToolExecute: func(ctx *HookContext) error {
            if cmd, ok := ctx.Data["command"]; ok {
                cmdStr := fmt.Sprintf("%v", cmd)
                for _, pattern := range p.blockedPatterns {
                    if matched := strings.Contains(cmdStr,
        pattern); matched {
                        ctx.Cancel = true
                        ctx.Error = fmt.Errorf("safety guard
        blocked: %s", pattern)
                        return ctx.Error
                    }
                }
            }
            return nil
        },
    }
}
```

MODIFY: `internal/session/session.go`

```
// In Session struct, add:
type Session struct {
    // ... existing fields ...
    PluginManager *plugins.PluginManager
    HookRegistry  *plugins.HookRegistry
}

// In NewSession(), add:
func NewSession(...) *Session {
    // ... existing initialization ...
    pm := plugins.NewPluginManager(nil)
    _ = pm.DiscoverAndLoad(ctx) // Best-effort
    s.PluginManager = pm
    s.HookRegistry = pm.GetRegistry()
```

```
    return s
}

// In every major lifecycle method, dispatch hooks:
func (s *Session) Start(ctx context.Context) error {
    hookCtx := plugins.NewHookContext(ctx, s.ID)
    if err :=
        s.HookRegistry.Dispatch(plugins.HookPreSessionStart,
        hookCtx); err != nil {
        return err
    }
    // ... existing logic ...
    return s.HookRegistry.Dispatch(plugins.HookPostSessionStart,
        hookCtx)
}
```

Anti-Bluff Test

```
# TEST 1: Verify all 25+ hook points are valid
go test ./internal/plugins/ -run
        TestHookRegistry_ValidateAllPoints -v

# TEST 2: Register a plugin, dispatch hook, verify handler called
go test ./internal/plugins/ -run TestHookRegistry_Dispatch -v

# TEST 3: Load built-in plugin via manager
go test ./internal/plugins/ -run TestPluginManager_LoadBuiltIn -v

# TEST 4: Verify safety guard cancels dangerous command
go test ./internal/plugins/ -run TestSafetyGuardPlugin_BlockRMRF
        -v

# TEST 5: Plugin manifest discovery in ~/.helix/plugins
go test ./internal/plugins/ -run TestPluginLoader_Discover -v

# TEST 6: End-to-end: plugin hook fires during session
go test ./internal/session/ -run TestSession_PluginHooks -v
```

Integration Verification

- [ ] All 29 hook points defined and dispatchable
- [ ] Built-in plugins (audit-log, token-counter, cost-tracker, safety-guard) load and execute
- [ ] `HookPreToolExecute` cancellation stops tool before execution
- [ ] `HookPostToolExecute` receives tool result data

- [ ] Plugin manifest JSON schema validated
- [ ] Go plugin `.so` loading works with exported `Plugin` symbol
- [ ] npm plugin bridge can execute Node.js subprocess

Feature 4: Local-First Architecture

Source Location (in original agent)

- `opencode/src/config/local-mode.ts` — Local-only settings
- `opencode/src/llm/local-providers.ts` — Ollama, LM Studio integration
- `opencode/src/privacy/no-telemetry.ts` — Telemetry disabling

Target Location (in HelixCode)

- `internal/llm/providers/ollama.go` (MODIFY/ENHANCE)
- `internal/llm/providers/local.go` (NEW)
- `internal/config/local_config.go` (NEW)
- `internal/privacy/telemetry.go` (NEW)
- `internal/llm/provider_registry.go` (MODIFY)

Exact Code Changes

NEW FILE: `internal/config/local_config.go`

```
package config

import (
    "fmt"
    "os"
)

// LocalConfig controls local-first behavior
type LocalConfig struct {
    Enabled           bool   `json:"enabled" yaml:"enabled"`
    DefaultToLocal    bool   `json:"default_to_local"
        yaml:"default_to_local"`
    AllowRemote       bool   `json:"allow_remote"
        yaml:"allow_remote"`
    RequireExplicit   bool   `json:"require_explicit"
        yaml:"require_explicit"`
    OllamaURL         string `json:"ollama_url"
        yaml:"ollama_url"`
    LMStudioURL       string `json:"lmstudio_url"
        yaml:"lmstudio_url"`
```

```
    LocalModelDir     string `json:"local_model_dir"
        yaml:"local_model_dir"`
    OfflineCacheDir   string `json:"offline_cache_dir"
        yaml:"offline_cache_dir"`
    NoTelemetry       bool   `json:"no_telemetry"
        yaml:"no_telemetry"`
    NoCloudSync       bool   `json:"no_cloud_sync"
        yaml:"no_cloud_sync"`
}

func DefaultLocalConfig() *LocalConfig {
    return &LocalConfig{
        Enabled:         true,
        DefaultToLocal:  false,
        AllowRemote:     true,
        RequireExplicit: false,
        OllamaURL:       "http://localhost:11434",
        LMStudioURL:     "http://localhost:1234/v1",
        LocalModelDir:   os.ExpandEnv("$HOME/.helix/models"),
        OfflineCacheDir: os.ExpandEnv("$HOME/.helix/cache"),
        NoTelemetry:     true,
        NoCloudSync:     true,
    }
}

// Validate ensures local config is consistent
func (lc *LocalConfig) Validate() error {
    if !lc.AllowRemote && !lc.DefaultToLocal {
        return fmt.Errorf("local-first: cannot disallow remote
        without defaulting to local")
    }
    return nil
}
```

NEW FILE: internal/privacy/telemetry.go

```
package privacy

import (
    "log"
    "os"
    "sync"
)

// TelemetryController manages data collection policies
type TelemetryController struct {
    disabled bool
    mu       sync.RWMutex
```

```
}

func NewTelemetryController() *TelemetryController {
    // Respect environment and config
    disabled := os.Getenv("HELIX_NO_TELEMETRY") == "1"
    return &TelemetryController{disabled: disabled}
}

func (tc *TelemetryController) Disable() {
    tc.mu.Lock()
    defer tc.mu.Unlock()
    tc.disabled = true
    log.Println("[Privacy] Telemetry disabled")
}

func (tc *TelemetryController) IsDisabled() bool {
    tc.mu.RLock()
    defer tc.mu.RUnlock()
    return tc.disabled
}

// SafeLog logs only if telemetry is not fully disabled (for
        critical errors)
func (tc *TelemetryController) SafeLog(msg string) {
    if !tc.disabled {
        log.Println(msg)
    }
}
```

MODIFY: `internal/llm/provider_registry.go` — **Add local model loading**

```
// Add method to ProviderRegistry:
func (reg *ProviderRegistry) LoadLocalModels() error {
    // Load Ollama models
    ollamaModels := reg.discoverOllamaModels()
    for _, m := range ollamaModels {
        m.IsLocal = true
        reg.models[m.ID] = m
    }

    // Load LM Studio models
    lmStudioModels := reg.discoverLMStudioModels()
    for _, m := range lmStudioModels {
        m.IsLocal = true
        reg.models[m.ID] = m
    }

    return nil
```

```
}

func (reg *ProviderRegistry) discoverOllamaModels()
        []*ModelDescriptor {
    // Call Ollama /api/tags endpoint
    // Parse response into ModelDescriptors
    return []*ModelDescriptor{} // Placeholder
}

func (reg *ProviderRegistry) discoverLMStudioModels()
        []*ModelDescriptor {
    // Call LM Studio /v1/models endpoint
    return []*ModelDescriptor{} // Placeholder
}
```

NEW FILE: internal/llm/providers/local.go

```
package providers

import (
    "context"
    "dev.helix.code/internal/llm"
    "time"
)

// LocalProvider wraps Ollama / LM Studio for fully local
        inference
type LocalProvider struct {
    ollama   llm.Provider
    lmStudio llm.Provider
    active   llm.Provider
}

func NewLocalProvider(ollama, lmStudio llm.Provider)
        *LocalProvider {
    return &LocalProvider{
        ollama:   ollama,
        lmStudio: lmStudio,
        active:   ollama, // Prefer Ollama
    }
}

func (p *LocalProvider) Generate(ctx context.Context, prompt
        string, opts ...llm.GenerateOption) (string, error) {
    return p.active.Generate(ctx, prompt, opts...)
}
```

```go
func (p *LocalProvider) GenerateStream(ctx context.Context,
        prompt string, opts ...llm.GenerateOption) (<-chan
        llm.StreamChunk, error) {
    return p.active.GenerateStream(ctx, prompt, opts...)
}

func (p *LocalProvider) GetModels() []*llm.ModelDescriptor {
    var all []*llm.ModelDescriptor
    if p.ollama != nil {
        all = append(all, p.ollama.GetModels()...)
    }
    if p.lmStudio != nil {
        all = append(all, p.lmStudio.GetModels()...)
    }
    for _, m := range all {
        m.IsLocal = true
        m.InputPricePer1K = 0
        m.OutputPricePer1K = 0
    }
    return all
}

func (p *LocalProvider) HealthCheck(ctx context.Context)
        (time.Duration, error) {
    return p.active.HealthCheck(ctx)
}

func (p *LocalProvider) SupportsModel(modelID string) bool {
    return p.active.SupportsModel(modelID)
}

func (p *LocalProvider) EstimateCost(_, _ int) float64 {
    return 0 // Local = free
}
```

Anti-Bluff Test

```
# TEST 1: Local config validates correctly
go test ./internal/config/ -run TestLocalConfig_Validate -v

# TEST 2: Telemetry controller respects env var
go test ./internal/privacy/ -run TestTelemetryController -v

# TEST 3: Local provider returns zero cost
go test ./internal/llm/providers/ -run
        TestLocalProvider_ZeroCost -v

# TEST 4: Offline mode with no remote providers configured
go test ./internal/llm/ -run TestProviderRouter_OfflineMode -v
```

```
# TEST 5: Ollama model discovery via /api/tags
go test ./internal/llm/ -run TestProviderRegistry_DiscoverOllama
        -v
```

Feature 5: Codebase Indexing

Source Location (in original agent)

- `opencode/src/indexing/vector-store.ts` — Vector database
- `opencode/src/indexing/embedder.ts` — Embedding generation
- `opencode/src/indexing/file-watcher.ts` — File change tracking
- `opencode/src/tools/grep.ts` — Semantic search tool

Target Location (in HelixCode)

- `internal/indexing/` (NEW directory)
  - `internal/indexing/indexer.go`
  - `internal/indexing/vector_store.go`
  - `internal/indexing/embedder.go`
  - `internal/indexing/file_watcher.go`
  - `internal/indexing/search_tool.go`
- `internal/tools/registry.go` (MODIFY)

Exact Code Changes

NEW FILE: `internal/indexing/vector_store.go`

```
package indexing

import (
    "fmt"
    "math"
    "sort"
    "sync"
)

// VectorStore is an in-memory vector database for code
        embeddings
type VectorStore struct {
    vectors map[string]*DocumentVector // filePath -> vector
    dim     int
    mu      sync.RWMutex
}

type DocumentVector struct {
    Path      string
```

```
    Content   string
    Embedding []float32
    Metadata  map[string]interface{}
}

type SearchResult struct {
    Path       string
    Score      float64
    Content    string
    Metadata   map[string]interface{}
}

func NewVectorStore(dim int) *VectorStore {
    return &VectorStore{
        vectors: make(map[string]*DocumentVector),
        dim:     dim,
    }
}

func (vs *VectorStore) Add(doc *DocumentVector) error {
    if len(doc.Embedding) != vs.dim {
        return fmt.Errorf("embedding dim %d != store dim %d",
        len(doc.Embedding), vs.dim)
    }
    vs.mu.Lock()
    defer vs.mu.Unlock()
    vs.vectors[doc.Path] = doc
    return nil
}

func (vs *VectorStore) Remove(path string) {
    vs.mu.Lock()
    defer vs.mu.Unlock()
    delete(vs.vectors, path)
}

func (vs *VectorStore) Search(query []float32, topK int)
        []SearchResult {
    vs.mu.RLock()
    defer vs.mu.RUnlock()

    if len(query) != vs.dim {
        return nil
    }

    var results []SearchResult
    for _, doc := range vs.vectors {
        score := cosineSimilarity(query, doc.Embedding)
```

```
            results = append(results, SearchResult{
                Path:    doc.Path,
                Score:   score,
                Content: doc.Content,
                Metadata: doc.Metadata,
            })
        }

        sort.Slice(results, func(i, j int) bool {
            return results[i].Score > results[j].Score
        })

        if len(results) > topK {
            results = results[:topK]
        }
        return results
    }

    func cosineSimilarity(a, b []float32) float64 {
        var dot, normA, normB float64
        for i := range a {
            dot += float64(a[i]) * float64(b[i])
            normA += float64(a[i]) * float64(a[i])
            normB += float64(b[i]) * float64(b[i])
        }
        if normA == 0 || normB == 0 {
            return 0
        }
        return dot / (math.Sqrt(normA) * math.Sqrt(normB))
    }
```

NEW FILE: internal/indexing/embedder.go

```
    package indexing

    import (
        "context"
        "dev.helix.code/internal/llm"
        "fmt"
    )

    // Embedder generates vector embeddings for code
    type Embedder struct {
        provider llm.Provider
    }

    func NewEmbedder(provider llm.Provider) *Embedder {
        return &Embedder{provider: provider}
```

```
}

func (e *Embedder) Embed(ctx context.Context, text string)
        ([]float32, error) {
    // Use provider's embedding endpoint if available
    // Fallback: use a lightweight local model
    return nil, fmt.Errorf("embedder not fully implemented")
}

func (e *Embedder) EmbedBatch(ctx context.Context, texts
        []string) ([][]float32, error) {
    results := make([][]float32, len(texts))
    for i, t := range texts {
        emb, err := e.Embed(ctx, t)
        if err != nil {
            return nil, err
        }
        results[i] = emb
    }
    return results, nil
}
```

NEW FILE: internal/indexing/indexer.go

```
package indexing

import (
    "context"
    "fmt"
    "log"
    "os"
    "path/filepath"
    "strings"
    "sync"

    "github.com/fsnotify/fsnotify"
)

// CodebaseIndexer manages the indexing lifecycle
type CodebaseIndexer struct {
    store      *VectorStore
    embedder   *Embedder
    watcher    *fsnotify.Watcher
    rootPath   string
    fileFilter func(string) bool
    mu         sync.RWMutex
    running    bool
}
```

```
func NewCodebaseIndexer(root string, embedder *Embedder)
        (*CodebaseIndexer, error) {
    watcher, err := fsnotify.NewWatcher()
    if err != nil {
        return nil, err
    }

    return &CodebaseIndexer{
        store:    NewVectorStore(768),
        embedder: embedder,
        watcher:  watcher,
        rootPath: root,
        fileFilter: func(path string) bool {
            // Exclude common non-code paths
            excluded := []string{
                ".git", "node_modules", "vendor", "dist",
        "build",
                ".cache", ".helix", "tmp", "temp",
            }
            for _, e := range excluded {
                if strings.Contains(path, e) {
                    return false
                }
            }
            // Include code extensions
            exts := []string{
                ".go", ".py", ".js", ".ts", ".java", ".rs",
        ".cpp", ".c",
                ".h", ".rb", ".php", ".swift", ".kt", ".scala",
                ".md", ".json", ".yaml", ".yml", ".toml",
            }
            for _, e := range exts {
                if strings.HasSuffix(path, e) {
                    return true
                }
            }
            return false
        },
    }, nil
}

// IndexAll walks the codebase and indexes all files
func (ci *CodebaseIndexer) IndexAll(ctx context.Context) error {
    ci.mu.Lock()
    ci.running = true
    ci.mu.Unlock()
```

```
    return filepath.Walk(ci.rootPath, func(path string, info
        os.FileInfo, err error) error {
        if err != nil {
            return err
        }
        if info.IsDir() {
            // Add directory to watcher
            _ = ci.watcher.Add(path)
            return nil
        }
        if !ci.fileFilter(path) {
            return nil
        }

        return ci.indexFile(ctx, path)
    })
}

func (ci *CodebaseIndexer) indexFile(ctx context.Context, path
        string) error {
    content, err := os.ReadFile(path)
    if err != nil {
        return err
    }

    embedding, err := ci.embedder.Embed(ctx, string(content))
    if err != nil {
        return err
    }

    doc := &DocumentVector{
        Path:      path,
        Content:   string(content),
        Embedding: embedding,
        Metadata: map[string]interface{}{
            "size": len(content),
            "ext":  filepath.Ext(path),
        },
    }

    return ci.store.Add(doc)
}

// Search performs semantic search across the indexed codebase
func (ci *CodebaseIndexer) Search(ctx context.Context, query
        string, topK int) ([]SearchResult, error) {
    queryEmbedding, err := ci.embedder.Embed(ctx, query)
    if err != nil {
```

```
        return nil, err
    }
    return ci.store.Search(queryEmbedding, topK), nil
}

// Watch starts the file watcher for real-time updates
func (ci *CodebaseIndexer) Watch(ctx context.Context) {
    for {
        select {
        case <-ctx.Done():
            return
        case event, ok := <-ci.watcher.Events:
            if !ok {
                return
            }
            if ci.fileFilter(event.Name) {
                if event.Op&fsnotify.Write == fsnotify.Write {
                    _ = ci.indexFile(ctx, event.Name)
                    log.Printf("[Indexer] Re-indexed: %s",
        event.Name)
                }
                if event.Op&fsnotify.Create == fsnotify.Create {
                    _ = ci.indexFile(ctx, event.Name)
                    log.Printf("[Indexer] Indexed new file: %s",
        event.Name)
                }
                if event.Op&fsnotify.Remove == fsnotify.Remove {
                    ci.store.Remove(event.Name)
                    log.Printf("[Indexer] Removed: %s",
        event.Name)
                }
            }
        case err, ok := <-ci.watcher.Errors:
            if !ok {
                return
            }
            log.Printf("[Indexer] Watcher error: %v", err)
        }
    }
}

// Stop halts the indexer
func (ci *CodebaseIndexer) Stop() {
    ci.watcher.Close()
}
```

NEW FILE: internal/indexing/search_tool.go

```go
package indexing

import (
    "context"
    "encoding/json"
    "fmt"
)

// SearchTool implements the `semantic_search` tool for the AI agent
type SearchTool struct {
    indexer *CodebaseIndexer
}

func NewSearchTool(indexer *CodebaseIndexer) *SearchTool {
    return &SearchTool{indexer: indexer}
}

func (st *SearchTool) Name() string        { return "semantic_search" }
func (st *SearchTool) Description() string { return "Search codebase using semantic similarity" }

type searchInput struct {
    Query string `json:"query"`
    TopK  int    `json:"top_k,omitempty"`
}

func (st *SearchTool) Execute(ctx context.Context, args json.RawMessage) (string, error) {
    var input searchInput
    if err := json.Unmarshal(args, &input); err != nil {
        return "", err
    }
    if input.TopK == 0 {
        input.TopK = 5
    }

    results, err := st.indexer.Search(ctx, input.Query, input.TopK)
    if err != nil {
        return "", err
    }

    output, _ := json.MarshalIndent(results, "", "  ")
    return string(output), nil
}
```

Anti-Bluff Test

```
# TEST 1: Vector store add and search
go test ./internal/indexing/ -run TestVectorStore_AddSearch -v

# TEST 2: File watcher detects changes
go test ./internal/indexing/ -run TestCodebaseIndexer_Watch -v

# TEST 3: Semantic search returns relevant results
go test ./internal/indexing/ -run TestSearchTool_Execute -v

# TEST 4: Cosine similarity computation
go test ./internal/indexing/ -run TestCosineSimilarity -v

# TEST 5: Full indexing of a test project
go test ./internal/indexing/ -run TestCodebaseIndexer_IndexAll -v
```

Feature 6: Multi-Modal Support

Source Location (in original agent)

- `opencode/src/llm/vision.ts` — Image input handling
- `opencode/src/llm/audio.ts` — Audio input handling
- `opencode/src/tools/attach.ts` — File attachment tool

Target Location (in HelixCode)

- `internal/llm/multimodal.go` (NEW)
- `internal/llm/image.go` (NEW)
- `internal/llm/audio.go` (NEW)
- `internal/tools/attach_tool.go` (NEW)

Exact Code Changes

NEW FILE: `internal/llm/multimodal.go`

```
package llm

import (
    "context"
    "encoding/base64"
    "fmt"
    "io"
    "mime"
    "os"
    "path/filepath"
)
```

```go
// ContentPart represents a single part of a multi-modal message
type ContentPart struct {
    Type     string `json:"type"` // "text", "image", "audio",
        "file"
    Text     string `json:"text,omitempty"`
    ImageURL string `json:"image_url,omitempty"`
    AudioURL string `json:"audio_url,omitempty"`
    FileData []byte `json:"-"` // raw bytes, not serialized
    MimeType string `json:"mime_type,omitempty"`
}

// MultiModalMessage is a message with multiple content parts
type MultiModalMessage struct {
    Role    string        `json:"role"`
    Content []ContentPart `json:"content"`
}

// MultiModalProvider extends Provider with multi-modal
        capabilities
type MultiModalProvider interface {
    Provider
    SupportsVision() bool
    SupportsAudio() bool
    GenerateMultiModal(ctx context.Context, messages
        []MultiModalMessage, opts ...GenerateOption) (string,
        error)
}

// EncodeImageBase64 encodes an image file to base64 data URI
func EncodeImageBase64(path string) (string, string, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return "", "", err
    }
    mimeType := mime.TypeByExtension(filepath.Ext(path))
    if mimeType == "" {
        mimeType = "image/png"
    }
    encoded := base64.StdEncoding.EncodeToString(data)
    dataURI := fmt.Sprintf("data:%s;base64,%s", mimeType,
        encoded)
    return dataURI, mimeType, nil
}

// EncodeAudioBase64 encodes an audio file to base64 data URI
func EncodeAudioBase64(path string) (string, string, error) {
    data, err := os.ReadFile(path)
```

```
    if err != nil {
        return "", "", err
    }
    mimeType := mime.TypeByExtension(filepath.Ext(path))
    if mimeType == "" {
        mimeType = "audio/mpeg"
    }
    encoded := base64.StdEncoding.EncodeToString(data)
    dataURI := fmt.Sprintf("data:%s;base64,%s", mimeType,
        encoded)
    return dataURI, mimeType, nil
}
```

NEW FILE: internal/tools/attach_tool.go

```
package tools

import (
    "context"
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
)

// AttachTool allows attaching files/images/audio to the
        conversation
type AttachTool struct {
    attachments []string // list of attached file paths
}

func NewAttachTool() *AttachTool {
    return &AttachTool{attachments: make([]string, 0)}
}

func (at *AttachTool) Name() string        { return "attach" }
func (at *AttachTool) Description() string { return "Attach a
        file, image, or audio to the conversation" }

type attachInput struct {
    Path string `json:"path"`
}

func (at *AttachTool) Execute(ctx context.Context, args
        json.RawMessage) (string, error) {
    var input attachInput
    if err := json.Unmarshal(args, &input); err != nil {
        return "", err
```

```
    }

    if _, err := os.Stat(input.Path); err != nil {
        return "", fmt.Errorf("file not found: %s", input.Path)
    }

    at.attachments = append(at.attachments, input.Path)

    ext := filepath.Ext(input.Path)
    mimeType := "unknown"
    switch ext {
    case ".png", ".jpg", ".jpeg", ".gif", ".webp", ".bmp":
        mimeType = "image"
    case ".mp3", ".wav", ".ogg", ".m4a", ".flac":
        mimeType = "audio"
    case ".txt", ".md", ".go", ".py", ".js", ".json", ".yaml",
         ".csv":
        mimeType = "text"
    default:
        mimeType = "file"
    }

    return fmt.Sprintf("Attached %s (%s)", input.Path,
        mimeType), nil
}

func (at *AttachTool) GetAttachments() []string {
    return at.attachments
}

func (at *AttachTool) ClearAttachments() {
    at.attachments = nil
}
```

Anti-Bluff Test

```
# TEST 1: Encode image to base64 data URI
go test ./internal/llm/ -run TestEncodeImageBase64 -v

# TEST 2: Attach tool accepts image path
go test ./internal/tools/ -run TestAttachTool_Image -v

# TEST 3: Multi-modal message serialization
go test ./internal/llm/ -run TestMultiModal_Message -v

# TEST 4: Vision capability detection on provider
go test ./internal/llm/ -run
        TestMultiModalProvider_SupportsVision -v
```

```
# TEST 5: Audio encoding
go test ./internal/llm/ -run TestEncodeAudioBase64 -v
```

Feature 7: Web Search Integration

Source Location (in original agent)

- `opencode/src/tools/websearch.ts` — Web search tool
- `opencode/src/tools/webfetch.ts` — URL fetching tool
- `opencode/src/search/result-summarizer.ts` — Result summarization

Target Location (in HelixCode)

- `internal/tools/web_search_tool.go` (NEW)
- `internal/tools/web_fetch_tool.go` (NEW)
- `internal/search/summarizer.go` (NEW)
- `internal/search/result.go` (NEW)

Exact Code Changes

NEW FILE: `internal/search/result.go`

```
package search

import (
    "time"
)

// SearchResult represents a single web search result
type SearchResult struct {
    Title       string    `json:"title"`
    URL         string    `json:"url"`
    Snippet     string    `json:"snippet"`
    Source      string    `json:"source"`
    PublishedAt time.Time `json:"published_at,omitempty"`
    Relevance   float64   `json:"relevance"`
}

// SearchEngine is the interface for search backends
type SearchEngine interface {
    Search(ctx context.Context, query string, maxResults int)
        ([]SearchResult, error)
}
```

NEW FILE: internal/tools/web_search_tool.go

```
package tools

import (
    "context"
    "encoding/json"
    "fmt"

    "dev.helix.code/internal/search"
)

// WebSearchTool performs real-time web searches
type WebSearchTool struct {
    engines []search.SearchEngine
}

func NewWebSearchTool(engines ...search.SearchEngine)
        *WebSearchTool {
    return &WebSearchTool{engines: engines}
}

func (wst *WebSearchTool) Name() string        { return
        "web_search" }
func (wst *WebSearchTool) Description() string { return "Search
        the web for current information" }

type webSearchInput struct {
    Query      string `json:"query"`
    MaxResults int    `json:"max_results,omitempty"`
}

func (wst *WebSearchTool) Execute(ctx context.Context, args
        json.RawMessage) (string, error) {
    var input webSearchInput
    if err := json.Unmarshal(args, &input); err != nil {
        return "", err
    }
    if input.MaxResults == 0 {
        input.MaxResults = 5
    }

    var allResults []search.SearchResult
    for _, engine := range wst.engines {
        results, err := engine.Search(ctx, input.Query,
        input.MaxResults)
        if err != nil {
            continue
        }
```

```
        allResults = append(allResults, results...)
    }

    if len(allResults) == 0 {
        return "No results found.", nil
    }

    output, _ := json.MarshalIndent(allResults, "", "  ")
    return string(output), nil
}
```

NEW FILE: internal/tools/web_fetch_tool.go

```
package tools

import (
    "context"
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "strings"
    "time"

    "github.com/PuerkitoBio/goquery"
)

// WebFetchTool fetches and extracts content from URLs
type WebFetchTool struct {
    client *http.Client
}

func NewWebFetchTool() *WebFetchTool {
    return &WebFetchTool{
        client: &http.Client{Timeout: 30 * time.Second},
    }
}

func (wft *WebFetchTool) Name() string        { return
        "web_fetch" }
func (wft *WebFetchTool) Description() string { return "Fetch
        content from a URL" }

type webFetchInput struct {
    URL    string `json:"url"`
    Format string `json:"format,omitempty"` // "text", "html",
        "markdown"
}
```

```
type webFetchOutput struct {
    URL     string `json:"url"`
    Title   string `json:"title"`
    Content string `json:"content"`
    Format  string `json:"format"`
}

func (wft *WebFetchTool) Execute(ctx context.Context, args
        json.RawMessage) (string, error) {
    var input webFetchInput
    if err := json.Unmarshal(args, &input); err != nil {
        return "", err
    }
    if input.Format == "" {
        input.Format = "text"
    }

    req, err := http.NewRequestWithContext(ctx, "GET",
        input.URL, nil)
    if err != nil {
        return "", err
    }
    req.Header.Set("User-Agent", "helix_code/1.0 WebFetch")

    resp, err := wft.client.Do(req)
    if err != nil {
        return "", err
    }
    defer resp.Body.Close()

    if resp.StatusCode != http.StatusOK {
        return "", fmt.Errorf("HTTP %d", resp.StatusCode)
    }

    body, err := io.ReadAll(io.LimitReader(resp.Body,
        1024*1024)) // 1MB limit
    if err != nil {
        return "", err
    }

    output := webFetchOutput{
        URL:    input.URL,
        Format: input.Format,
    }

    switch input.Format {
    case "html":
        output.Content = string(body)
```

```
            output.Title = extractTitle(string(body))
        case "markdown":
            output.Content = htmlToMarkdown(string(body))
            output.Title = extractTitle(string(body))
        default: // text
            output.Content = extractText(string(body))
            output.Title = extractTitle(string(body))
        }

        result, _ := json.MarshalIndent(output, "", "  ")
        return string(result), nil
    }

    func extractTitle(html string) string {
        doc, err :=
            goquery.NewDocumentFromReader(strings.NewReader(html))
        if err != nil {
            return ""
        }
        return doc.Find("title").First().Text()
    }

    func extractText(html string) string {
        doc, err :=
            goquery.NewDocumentFromReader(strings.NewReader(html))
        if err != nil {
            return html
        }
        return doc.Find("body").Text()
    }

    func htmlToMarkdown(html string) string {
        // Simplified conversion: strip tags, keep structure hints
        text := extractText(html)
        return text
    }
```

NEW FILE: internal/search/summarizer.go

```
    package search

    import (
        "context"
        "fmt"
        "strings"

        "dev.helix.code/internal/llm"
    )
```

```go
// ResultSummarizer uses an LLM to summarize search results
type ResultSummarizer struct {
    provider llm.Provider
}

func NewResultSummarizer(provider llm.Provider)
        *ResultSummarizer {
    return &ResultSummarizer{provider: provider}
}

// Summarize condenses search results into a coherent answer
func (rs *ResultSummarizer) Summarize(ctx context.Context, query
        string, results []SearchResult) (string, error) {
    var sb strings.Builder
    sb.WriteString(fmt.Sprintf("Summarize the following search
        results for the query: %q\n\n", query))
    for i, r := range results {
        sb.WriteString(fmt.Sprintf("[%d] %s\nURL: %s\n%s\n\n",
        i+1, r.Title, r.URL, r.Snippet))
    }
    sb.WriteString("Provide a concise, factual summary with
        source attribution.")

    return rs.provider.Generate(ctx, sb.String())
}
```

Anti-Bluff Test

```
# TEST 1: Web fetch extracts title and text
go test ./internal/tools/ -run TestWebFetchTool_Execute -v

# TEST 2: Search engine interface works with mock
go test ./internal/search/ -run TestSearchEngine_Mock -v

# TEST 3: Summarizer builds correct prompt
go test ./internal/search/ -run TestSummarizer_Prompt -v

# TEST 4: Web fetch respects 1MB limit
go test ./internal/tools/ -run TestWebFetchTool_SizeLimit -v

# TEST 5: Search tool returns JSON results
go test ./internal/tools/ -run TestWebSearchTool_Execute -v
```

Feature 8: Custom Commands

Source Location (in original agent)

- `opencode/src/commands/custom-command.ts` — Custom command parser
- `opencode/src/commands/command-template.ts` — Template engine
- `opencode/src/commands/batch.ts` — Batch execution

Target Location (in HelixCode)

- `internal/commands/` (NEW directory)
  - `internal/commands/custom_command.go`
  - `internal/commands/template.go`
  - `internal/commands/registry.go`
  - `internal/commands/batch.go`
- `cmd/cli/main.go` (MODIFY)

Exact Code Changes

NEW FILE: `internal/commands/custom_command.go`

```
package commands

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "regexp"
    "strings"
)

// CustomCommand is a user-defined command
type CustomCommand struct {
    Name        string            `json:"name" yaml:"name"`
    Description string            `json:"description"
        yaml:"description"`
    Prompt      string            `json:"prompt"
        yaml:"prompt"`       // Template string
    Args        []CommandArg      `json:"args"
        yaml:"args"`           // Named placeholders
    Model       string            `json:"model,omitempty"
        yaml:"model,omitempty"`
    Tools       []string          `json:"tools,omitempty"
        yaml:"tools,omitempty"`   // Allowed tools
    Permission  string            `json:"permission,omitempty"
        yaml:"permission,omitempty"`
}

type CommandArg struct {
    Name        string `json:"name" yaml:"name"`
```

```
    Type        string `json:"type"
        yaml:"type"`                // "string", "number",
        "boolean"
    Required    bool   `json:"required" yaml:"required"`
    Default     string `json:"default,omitempty"
        yaml:"default,omitempty"`
    Description string `json:"description,omitempty"
        yaml:"description,omitempty"`
}

// ParseArgs extracts named arguments from user input
func (cc *CustomCommand) ParseArgs(input string)
        (map[string]string, error) {
    result := make(map[string]string)

    // Extract quoted and unquoted arguments
    re := regexp.MustCompile(`"([^"]+)"|\S+`)
    matches := re.FindAllStringSubmatch(input, -1)
    var rawArgs []string
    for _, m := range matches {
        if m[1] != "" {
            rawArgs = append(rawArgs, m[1])
        } else {
            rawArgs = append(rawArgs, m[0])
        }
    }

    // Map positional args to named args
    for i, argDef := range cc.Args {
        if i < len(rawArgs) {
            result[argDef.Name] = rawArgs[i]
        } else if argDef.Required {
            return nil, fmt.Errorf("missing required argument:
        %s", argDef.Name)
        } else if argDef.Default != "" {
            result[argDef.Name] = argDef.Default
        }
    }

    return result, nil
}

// Render builds the final prompt from template + args
func (cc *CustomCommand) Render(args map[string]string) string {
    prompt := cc.Prompt
    for k, v := range args {
        placeholder := fmt.Sprintf("{{%s}}", k)
        prompt = strings.ReplaceAll(prompt, placeholder, v)
```

```
    }
    return prompt
}
```

NEW FILE: internal/commands/registry.go

```
package commands

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "sync"
)

// CommandRegistry stores all custom commands
type CommandRegistry struct {
    commands map[string]*CustomCommand
    mu       sync.RWMutex
    path     string
}

func NewCommandRegistry(path string) *CommandRegistry {
    return &CommandRegistry{
        commands: make(map[string]*CustomCommand),
        path:     path,
    }
}

// Load discovers commands from ~/.helix/commands/
func (cr *CommandRegistry) Load() error {
    entries, err := os.ReadDir(cr.path)
    if err != nil {
        if os.IsNotExist(err) {
            return nil
        }
        return err
    }

    for _, entry := range entries {
        if entry.IsDir() {
            continue
        }
        ext := strings.ToLower(filepath.Ext(entry.Name()))
        if ext != ".json" && ext != ".yaml" && ext != ".md" {
            continue
```

```
        }

        cmd, err := cr.loadFile(filepath.Join(cr.path,
        entry.Name()))
        if err != nil {
            continue
        }
        cr.commands[cmd.Name] = cmd
    }
    return nil
}

func (cr *CommandRegistry) loadFile(path string)
        (*CustomCommand, error) {
    data, err := os.ReadFile(path)
    if err != nil {
        return nil, err
    }

    if strings.HasSuffix(path, ".md") {
        return cr.parseMarkdownCommand(data)
    }

    var cmd CustomCommand
    if err := json.Unmarshal(data, &cmd); err != nil {
        return nil, err
    }
    return &cmd, nil
}

// parseMarkdownCommand extracts command from markdown
        frontmatter
func (cr *CommandRegistry) parseMarkdownCommand(data []byte)
        (*CustomCommand, error) {
    content := string(data)
    // Extract YAML frontmatter between ---
    if !strings.HasPrefix(content, "---") {
        return nil, fmt.Errorf("no frontmatter found")
    }
    end := strings.Index(content[3:], "---")
    if end == -1 {
        return nil, fmt.Errorf("frontmatter not closed")
    }
    frontmatter := content[3 : end+3]

    var cmd CustomCommand
    // Use goccy/go-yaml or similar to parse frontmatter
    _ = frontmatter
```

```
    return &cmd, fmt.Errorf("YAML frontmatter parsing requires
        yaml package")
}

// Get returns a command by name
func (cr *CommandRegistry) Get(name string) (*CustomCommand,
        bool) {
    cr.mu.RLock()
    defer cr.mu.RUnlock()
    c, ok := cr.commands[name]
    return c, ok
}

// List returns all commands
func (cr *CommandRegistry) List() []*CustomCommand {
    cr.mu.RLock()
    defer cr.mu.RUnlock()
    result := make([]*CustomCommand, 0, len(cr.commands))
    for _, c := range cr.commands {
        result = append(result, c)
    }
    return result
}

// Add registers a new command
func (cr *CommandRegistry) Add(cmd *CustomCommand) error {
    cr.mu.Lock()
    defer cr.mu.Unlock()
    if _, exists := cr.commands[cmd.Name]; exists {
        return fmt.Errorf("command %s already exists", cmd.Name)
    }
    cr.commands[cmd.Name] = cmd
    return cr.save()
}

func (cr *CommandRegistry) save() error {
    data, err := json.MarshalIndent(cr.commands, "", "  ")
    if err != nil {
        return err
    }
    return os.WriteFile(filepath.Join(cr.path,
        "_registry.json"), data, 0644)
}
```

NEW FILE: internal/commands/batch.go

```
package commands
```

```
import (
    "context"
    "dev.helix.code/internal/llm"
    "fmt"
    "sync"
)

// BatchExecutor runs multiple commands in parallel
type BatchExecutor struct {
    provider llm.Provider
}

func NewBatchExecutor(provider llm.Provider) *BatchExecutor {
    return &BatchExecutor{provider: provider}
}

// BatchItem is a single command in a batch
type BatchItem struct {
    Command string
    Args    map[string]string
    Result  string
    Error   error
}

// ExecuteBatch runs custom commands in parallel
func (be *BatchExecutor) ExecuteBatch(ctx context.Context,
        registry *CommandRegistry, items []BatchItem)
        []BatchItem {
    var wg sync.WaitGroup
    results := make([]BatchItem, len(items))

    for i, item := range items {
        wg.Add(1)
        go func(idx int, it BatchItem) {
            defer wg.Done()
            cmd, ok := registry.Get(it.Command)
            if !ok {
                results[idx] = BatchItem{
                    Command: it.Command,
                    Error:   fmt.Errorf("command not found: %s",
        it.Command),
                }
                return
            }

            prompt := cmd.Render(it.Args)
            result, err := be.provider.Generate(ctx, prompt)
            results[idx] = BatchItem{
```

```
                Command: it.Command,
                Args:    it.Args,
                Result:  result,
                Error:   err,
            }
        }(i, item)
    }

    wg.Wait()
    return results
}
```

MODIFY: `cmd/cli/main.go` — **Add custom command handling**

```
// Add flag:
customCommand := flag.String("cmd", "", "Run a custom command")
customArgs := flag.String("cmd-args", "", "Arguments for custom
        command")

// Add case:
case *customCommand != "":
    return c.handleCustomCommand(ctx, *customCommand,
        *customArgs)

// Add handler:
func (c *CLI) handleCustomCommand(ctx context.Context, name,
        args string) error {
    registry :=
        commands.NewCommandRegistry(os.ExpandEnv("$HOME/.helix/
        commands"))
    _ = registry.Load()

    cmd, ok := registry.Get(name)
    if !ok {
        return fmt.Errorf("custom command %s not found", name)
    }

    parsedArgs, err := cmd.ParseArgs(args)
    if err != nil {
        return err
    }

    prompt := cmd.Render(parsedArgs)
    result, err := c.llmProvider.Generate(ctx, prompt)
    if err != nil {
        return err
    }
```

```
    fmt.Println(result)
    return nil
}
```

Anti-Bluff Test

```
# TEST 1: Parse named arguments
go test ./internal/commands/ -run TestCustomCommand_ParseArgs -v

# TEST 2: Render template with placeholders
go test ./internal/commands/ -run TestCustomCommand_Render -v

# TEST 3: Load commands from directory
go test ./internal/commands/ -run TestCommandRegistry_Load -v

# TEST 4: Batch execution parallelism
go test ./internal/commands/ -run TestBatchExecutor -v

# TEST 5: End-to-end: custom command from CLI
go test ./cmd/cli/ -run TestCLICustomCommand -v
```

Feature 9: Context Management

Source Location (in original agent)

- `opencode/src/context/token-counter.ts` — Token counting
- `opencode/src/context/truncator.ts` — Smart truncation
- `opencode/src/context/priority.ts` — Priority-based retention
- `opencode/src/context/compactor.ts` — Context compaction

Target Location (in HelixCode)

- `internal/context/` (NEW directory)
  - `internal/context/token_counter.go`
  - `internal/context/truncator.go`
  - `internal/context/priority.go`
  - `internal/context/manager.go`
- `internal/llm/prompt_builder.go` (MODIFY)
- `internal/session/session.go` (MODIFY)

Exact Code Changes

NEW FILE: `internal/context/token_counter.go`

```
package context

import (
    "strings"
```

```
    "unicode/utf8"
)

// TokenCounter estimates token counts using multiple strategies
type TokenCounter struct {
    method CountMethod
}

type CountMethod string

const (
    MethodChar4      CountMethod = "char4"      // 1 token ≈ 4
        chars
    MethodWord15     CountMethod = "word15"     // 1 word ≈ 1.5
        tokens
    MethodWhitespace CountMethod = "whitespace" // Split on
        whitespace
)

func NewTokenCounter(method CountMethod) *TokenCounter {
    if method == "" {
        method = MethodChar4
    }
    return &TokenCounter{method: method}
}

// Count estimates tokens in text
func (tc *TokenCounter) Count(text string) int {
    switch tc.method {
    case MethodChar4:
        return utf8.RuneCountInString(text) / 4
    case MethodWord15:
        words := len(strings.Fields(text))
        return int(float64(words) * 1.5)
    case MethodWhitespace:
        return len(strings.Fields(text))
    default:
        return utf8.RuneCountInString(text) / 4
    }
}

// CountMessages estimates tokens for a list of messages
func (tc *TokenCounter) CountMessages(messages []Message) int {
    total := 0
    for _, m := range messages {
        total += tc.Count(m.Role) + tc.Count(m.Content)
        // Add overhead per message
        total += 4
```

```
    }
    return total
}

type Message struct {
    Role    string
    Content string
}
```

NEW FILE: internal/context/priority.go

```
package context

// PriorityLevel defines how important a message is for retention
type PriorityLevel int

const (
    PrioritySystem      PriorityLevel = 100 // System prompt —
        never drop
    PriorityToolResult  PriorityLevel = 90  // Recent tool
        results
    PriorityUserQuery   PriorityLevel = 80  // Current user query
    PriorityAssistant   PriorityLevel = 70  // Recent assistant
        responses
    PriorityContextFile PriorityLevel = 60  // Referenced files
    PriorityHistory     PriorityLevel = 40  // Older conversation
    PriorityObsolete    PriorityLevel = 10  // Outdated
        information
)

// PrioritizedMessage wraps a message with priority
type PrioritizedMessage struct {
    Message
    Priority  PriorityLevel
    Timestamp int64
    Tokens    int
}

// ByPriority sorts messages by priority descending, then
        timestamp ascending
func ByPriority(a, b PrioritizedMessage) bool {
    if a.Priority != b.Priority {
        return a.Priority > b.Priority
    }
    return a.Timestamp < b.Timestamp
}
```

NEW FILE: internal/context/truncator.go

```go
package context

import (
    "sort"
    "time"
)

// Truncator trims context to fit within token budget
type Truncator struct {
    counter *TokenCounter
}

func NewTruncator(counter *TokenCounter) *Truncator {
    return &Truncator{counter: counter}
}

// Truncate reduces messages to fit within maxTokens using
        priority
func (t *Truncator) Truncate(messages []PrioritizedMessage,
        maxTokens int) []PrioritizedMessage {
    // Calculate current token count
    current := 0
    for _, m := range messages {
        m.Tokens = t.counter.Count(m.Content) +
        t.counter.Count(m.Role) + 4
        current += m.Tokens
    }

    if current <= maxTokens {
        return messages
    }

    // Sort by priority (high first), then by age (old first to
        drop)
    sort.SliceStable(messages, func(i, j int) bool {
        if messages[i].Priority != messages[j].Priority {
            return messages[i].Priority > messages[j].Priority
        }
        return messages[i].Timestamp > messages[j].Timestamp
    })

    // Drop lowest priority messages until within budget
    var result []PrioritizedMessage
    remaining := maxTokens
    for _, m := range messages {
        if m.Tokens <= remaining {
            result = append(result, m)
```

```
            remaining -= m.Tokens
        } else if m.Priority == PrioritySystem {
            // System messages are non-negotiable — compress
        instead
            compressed := t.compressMessage(m, remaining-4)
            result = append(result, compressed)
            remaining = 0
        }
        if remaining <= 0 {
            break
        }
    }

    // Re-sort by timestamp for correct ordering
    sort.SliceStable(result, func(i, j int) bool {
        return result[i].Timestamp < result[j].Timestamp
    })

    return result
}

// compressMessage reduces a message to target token count
func (t *Truncator) compressMessage(m PrioritizedMessage,
        targetTokens int) PrioritizedMessage {
    // Simple compression: truncate content
    chars := targetTokens * 4
    if len(m.Content) > chars {
        m.Content = m.Content[:chars-3] + "..."
        m.Tokens = t.counter.Count(m.Content) +
        t.counter.Count(m.Role) + 4
    }
    return m
}
```

NEW FILE: internal/context/manager.go

```
package context

import (
    "context"
    "fmt"
    "sync"
    "time"
)

// Manager coordinates token counting, priority, and truncation
type Manager struct {
    counter    *TokenCounter
```

```
    truncator  *Truncator
    maxTokens  int
    messages   []PrioritizedMessage
    mu         sync.RWMutex
}

func NewManager(maxTokens int) *Manager {
    tc := NewTokenCounter(MethodChar4)
    return &Manager{
        counter:   tc,
        truncator: NewTruncator(tc),
        maxTokens: maxTokens,
        messages:  make([]PrioritizedMessage, 0),
    }
}

// AddMessage adds a message with automatic context management
func (cm *Manager) AddMessage(role, content string, priority
        PriorityLevel) {
    cm.mu.Lock()
    defer cm.mu.Unlock()

    msg := PrioritizedMessage{
        Message:   Message{Role: role, Content: content},
        Priority:  priority,
        Timestamp: time.Now().Unix(),
    }
    msg.Tokens = cm.counter.Count(content) +
        cm.counter.Count(role) + 4

    cm.messages = append(cm.messages, msg)

    // Auto-truncate if over budget
    current := cm.currentTokens()
    if current > cm.maxTokens {
        cm.messages = cm.truncator.Truncate(cm.messages,
        cm.maxTokens)
    }
}

// GetMessages returns the current context window
func (cm *Manager) GetMessages() []PrioritizedMessage {
    cm.mu.RLock()
    defer cm.mu.RUnlock()
    result := make([]PrioritizedMessage, len(cm.messages))
    copy(result, cm.messages)
    return result
}
```

```
// GetTokenCount returns current token usage
func (cm *Manager) GetTokenCount() int {
    cm.mu.RLock()
    defer cm.mu.RUnlock()
    return cm.currentTokens()
}

// Compact summarizes old messages into a single context message
func (cm *Manager) Compact(ctx context.Context, provider
        llm.Provider) error {
    cm.mu.Lock()
    defer cm.mu.Unlock()

    // Find old low-priority messages
    var toCompact []PrioritizedMessage
    var toKeep []PrioritizedMessage
    cutoff := time.Now().Add(-10 * time.Minute).Unix()

    for _, m := range cm.messages {
        if m.Timestamp < cutoff && m.Priority <= PriorityHistory
        {
            toCompact = append(toCompact, m)
        } else {
            toKeep = append(toKeep, m)
        }
    }

    if len(toCompact) == 0 {
        return nil
    }

    // Generate summary
    var content strings.Builder
    for _, m := range toCompact {
        content.WriteString(fmt.Sprintf("%s: %s\n", m.Role,
        m.Content))
    }

    summary, err := provider.Generate(ctx, "Summarize this
        conversation context concisely:\n"+content.String())
    if err != nil {
        return err
    }

    // Replace old messages with summary
    cm.messages = append([]PrioritizedMessage{{
        Message:   Message{Role: "system", Content: "[Context
        summary]: " + summary},
```

```
            Priority:  PrioritySystem,
            Timestamp: time.Now().Unix(),
        }}, toKeep...)

        return nil
}

func (cm *Manager) currentTokens() int {
        total := 0
        for _, m := range cm.messages {
            total += m.Tokens
        }
        return total
}

// Clear removes all messages
func (cm *Manager) Clear() {
        cm.mu.Lock()
        defer cm.mu.Unlock()
        cm.messages = nil
}
```

Anti-Bluff Test

```
# TEST 1: Token counter estimates correctly
go test ./internal/context/ -run TestTokenCounter_Count -v

# TEST 2: Truncator respects priority
go test ./internal/context/ -run TestTruncator_PriorityRetention
        -v

# TEST 3: Context manager auto-truncates
go test ./internal/context/ -run TestManager_AutoTruncate -v

# TEST 4: Compaction reduces token count
go test ./internal/context/ -run TestManager_Compact -v

# TEST 5: System messages survive truncation
go test ./internal/context/ -run TestTruncator_SystemSurvives -v
```

Feature 10: Performance Optimization

Source Location (in original agent)

- `opencode/src/perf/batcher.ts` — Request batching
- `opencode/src/perf/cache.ts` — Response caching
- `opencode/src/perf/parallel.ts` — Parallel execution

Target Location (in HelixCode)

- `internal/perf/` (NEW directory)
  - `internal/perf/batcher.go`
  - `internal/perf/cache.go`
  - `internal/perf/parallel.go`
- `internal/llm/provider_router.go` (MODIFY)
- `internal/tools/registry.go` (MODIFY)

Exact Code Changes

NEW FILE: `internal/perf/batcher.go`

```
package perf

import (
    "context"
    "sync"
    "time"
)

// RequestBatcher groups multiple requests into batches
type RequestBatcher struct {
    maxSize   int
    maxWait   time.Duration
    batchFn   func([]interface{}) ([]interface{}, error)
    pending   []batchItem
    mu        sync.Mutex
    timer     *time.Timer
}

type batchItem struct {
    request  interface{}
    response chan interface{}
    err      chan error
}

func NewRequestBatcher(maxSize int, maxWait time.Duration,
        batchFn func([]interface{}) ([]interface{}, error))
        *RequestBatcher {
    return &RequestBatcher{
        maxSize: maxSize,
        maxWait: maxWait,
        batchFn: batchFn,
        pending: make([]batchItem, 0),
    }
}

// Submit adds a request to the batch
```

```
func (rb *RequestBatcher) Submit(ctx context.Context, req
        interface{}) (interface{}, error) {
    item := batchItem{
        request:  req,
        response: make(chan interface{}, 1),
        err:      make(chan error, 1),
    }

    rb.mu.Lock()
    rb.pending = append(rb.pending, item)
    shouldFlush := len(rb.pending) >= rb.maxSize
    if rb.timer == nil {
        rb.timer = time.AfterFunc(rb.maxWait, rb.flush)
    }
    rb.mu.Unlock()

    if shouldFlush {
        rb.flush()
    }

    select {
    case <-ctx.Done():
        return nil, ctx.Err()
    case resp := <-item.response:
        return resp, nil
    case err := <-item.err:
        return nil, err
    }
}

func (rb *RequestBatcher) flush() {
    rb.mu.Lock()
    items := rb.pending
    rb.pending = make([]batchItem, 0)
    if rb.timer != nil {
        rb.timer.Stop()
        rb.timer = nil
    }
    rb.mu.Unlock()

    if len(items) == 0 {
        return
    }

    requests := make([]interface{}, len(items))
    for i, item := range items {
        requests[i] = item.request
    }
```

```
    responses, err := rb.batchFn(requests)
    if err != nil {
        for _, item := range items {
            item.err <- err
        }
        return
    }

    for i, item := range items {
        if i < len(responses) {
            item.response <- responses[i]
        } else {
            item.err <- fmt.Errorf("no response for request %d",
        i)
        }
    }
}
```

NEW FILE: internal/perf/cache.go

```
package perf

import (
    "crypto/sha256"
    "encoding/hex"
    "fmt"
    "time"

    "github.com/hashicorp/golang-lru/v2/expirable"
)

// ResponseCache caches LLM responses by prompt hash
type ResponseCache struct {
    cache *expirable.LRU[string, *CachedResponse]
}

type CachedResponse struct {
    Content   string
    CreatedAt time.Time
    TTL       time.Duration
    Hits      int
}

func NewResponseCache(maxEntries int, ttl time.Duration)
        *ResponseCache {
    return &ResponseCache{
```

```
        cache: expirable.NewLRU[string, *CachedResponse]
        (maxEntries, nil, ttl),
    }
}

// hashKey creates a deterministic key from prompt + model +
        params
func (rc *ResponseCache) hashKey(prompt, modelID string, temp
        float64) string {
    h := sha256.New()
    fmt.Fprintf(h, "%s|%s|%f", prompt, modelID, temp)
    return hex.EncodeToString(h.Sum(nil))
}

// Get retrieves a cached response
func (rc *ResponseCache) Get(prompt, modelID string, temp
        float64) (*CachedResponse, bool) {
    key := rc.hashKey(prompt, modelID, temp)
    resp, ok := rc.cache.Get(key)
    if ok && time.Since(resp.CreatedAt) < resp.TTL {
        resp.Hits++
        return resp, true
    }
    return nil, false
}

// Set stores a response in cache
func (rc *ResponseCache) Set(prompt, modelID string, temp
        float64, content string, ttl time.Duration) {
    key := rc.hashKey(prompt, modelID, temp)
    rc.cache.Add(key, &CachedResponse{
        Content:   content,
        CreatedAt: time.Now(),
        TTL:       ttl,
        Hits:      0,
    })
}

// Invalidate clears all cached entries
func (rc *ResponseCache) Invalidate() {
    // LRU does not support clear; create new instance
    rc.cache = expirable.NewLRU[string, *CachedResponse]
        (rc.cache.Len(), nil, time.Hour)
}
```

NEW FILE: internal/perf/parallel.go

```go
package perf

import (
    "context"
    "sync"
)

// ParallelExecutor runs tasks concurrently with a worker pool
type ParallelExecutor struct {
    workers int
}

func NewParallelExecutor(workers int) *ParallelExecutor {
    if workers <= 0 {
        workers = 4
    }
    return &ParallelExecutor{workers: workers}
}

// Task is a unit of work
type Task struct {
    ID   string
    Fn   func(context.Context) (interface{}, error)
}

// TaskResult is the outcome of a task
type TaskResult struct {
    ID     string
    Result interface{}
    Error  error
}

// Execute runs all tasks in parallel, bounded by worker count
func (pe *ParallelExecutor) Execute(ctx context.Context, tasks
        []Task) []TaskResult {
    results := make([]TaskResult, len(tasks))
    var wg sync.WaitGroup
    semaphore := make(chan struct{}, pe.workers)

    for i, task := range tasks {
        wg.Add(1)
        go func(idx int, t Task) {
            defer wg.Done()
            semaphore <- struct{}{}
            defer func() { <-semaphore }()

            res, err := t.Fn(ctx)
```

```
            results[idx] = TaskResult{
                ID:     t.ID,
                Result: res,
                Error:  err,
            }
        }(i, task)
    }

    wg.Wait()
    return results
}

// ExecuteFirstSuccess runs tasks in parallel and returns the
        first successful result
func (pe *ParallelExecutor) ExecuteFirstSuccess(ctx
        context.Context, tasks []Task) (interface{}, error) {
    ctx, cancel := context.WithCancel(ctx)
    defer cancel()

    resultCh := make(chan TaskResult, len(tasks))
    var wg sync.WaitGroup

    for _, task := range tasks {
        wg.Add(1)
        go func(t Task) {
            defer wg.Done()
            res, err := t.Fn(ctx)
            resultCh <- TaskResult{ID: t.ID, Result: res, Error:
        err}
            if err == nil {
                cancel() // Stop other tasks
            }
        }(task)
    }

    go func() {
        wg.Wait()
        close(resultCh)
    }()

    for r := range resultCh {
        if r.Error == nil {
            return r.Result, nil
        }
    }
    return nil, fmt.Errorf("all parallel tasks failed")
}
```

MODIFY: `internal/llm/provider_router.go` — **Add caching layer**

```
// In ProviderRouter struct, add:
type ProviderRouter struct {
    // ... existing fields ...
    cache *perf.ResponseCache
}

// In constructor:
func NewProviderRouter(registry *ProviderRegistry, strategy
        RouterStrategy) *ProviderRouter {
    return &ProviderRouter{
        // ... existing ...
        cache: perf.NewResponseCache(1000, 5*time.Minute),
    }
}

// In Route() or before LLM call, check cache:
func (pr *ProviderRouter) GenerateCached(ctx context.Context,
        sessionID, prompt, modelID string, temp float64)
        (string, error) {
    if cached, ok := pr.cache.Get(prompt, modelID, temp); ok {
        log.Printf("[ProviderRouter] Cache hit for model=%s",
        modelID)
        return cached.Content, nil
    }

    provider, model, err := pr.Route(ctx, sessionID, nil,
        modelID)
    if err != nil {
        return "", err
    }

    result, err := provider.Generate(ctx, prompt)
    if err != nil {
        return "", err
    }

    pr.cache.Set(prompt, modelID, temp, result, 5*time.Minute)
    return result, nil
}
```

Anti-Bluff Test

```
# TEST 1: Batcher groups multiple requests
go test ./internal/perf/ -run TestRequestBatcher_Batch -v

# TEST 2: Cache hit returns without LLM call
go test ./internal/perf/ -run TestResponseCache_HitMiss -v
```

```
# TEST 3: Parallel executor runs bounded workers
go test ./internal/perf/ -run TestParallelExecutor_Bounded -v

# TEST 4: ExecuteFirstSuccess cancels remaining tasks
go test ./internal/perf/ -run TestParallelExecutor_FirstSuccess
        -v

# TEST 5: Cache invalidation clears all entries
go test ./internal/perf/ -run TestResponseCache_Invalidate -v
```

Global Integration Verification

Build Verification

```
cd /mnt/agents/output/helixcode_sources

# 1. Add all new files to module
go mod tidy

# 2. Build all new packages
go build ./internal/llm/...
go build ./internal/tools/...
go build ./internal/plugins/...
go build ./internal/indexing/...
go build ./internal/context/...
go build ./internal/perf/...
go build ./internal/commands/...
go build ./internal/privacy/...
go build ./internal/search/...

# 3. Build main binary
go build -o helixcode ./main.go ./cmd_cli_main.go ./
        cmd_server_main.go

# 4. Run all tests
go test ./... -v
```

Dependency Additions to `go.mod`

```
// Already present:
// github.com/PuerkitoBio/goquery v1.10.3
// github.com/hashicorp/golang-lru/v2 v2.0.7
// github.com/fsnotify/fsnotify v1.9.0
// github.com/goccy/go-yaml v1.18.0
```

Configuration Integration (`internal/config/config.go`)

Add to existing Config struct:

```
type Config struct {
    // ... existing fields ...

    // NEW: OpenCode-ported features
    LSP        *LSPConfig        `json:"lsp,omitempty"
        yaml:"lsp,omitempty"`
    Plugins    *PluginsConfig    `json:"plugins,omitempty"
        yaml:"plugins,omitempty"`
    Local      *LocalConfig      `json:"local,omitempty"
        yaml:"local,omitempty"`
    Indexing   *IndexingConfig   `json:"indexing,omitempty"
        yaml:"indexing,omitempty"`
    Commands   *CommandsConfig   `json:"commands,omitempty"
        yaml:"commands,omitempty"`
}

type LSPConfig struct {
    Servers map[string]tools.LSPServerConfig `json:"servers"
        yaml:"servers"`
}

type PluginsConfig struct {
    Enabled     bool     `json:"enabled" yaml:"enabled"`
    SearchPaths []string `json:"search_paths"
        yaml:"search_paths"`
}

type IndexingConfig struct {
    Enabled  bool   `json:"enabled" yaml:"enabled"`
    Embedder string `json:"embedder" yaml:"embedder"`
}

type CommandsConfig struct {
    Path string `json:"path" yaml:"path"`
}
```

Complete File Inventory

New Files Created (48 files)

1. `internal/llm/model_descriptor.go`
2. `internal/llm/provider_router.go`
3. `internal/llm/provider_registry.go`
4. `internal/llm/health_monitor.go`

5. internal/llm/provider_switcher.go
6. internal/llm/multimodal.go
7. internal/llm/image.go
8. internal/llm/audio.go
9. internal/llm/providers/openai.go
10. internal/llm/providers/anthropic.go
11. internal/llm/providers/gemini.go
12. internal/llm/providers/ollama.go
13. internal/llm/providers/bedrock.go
14. internal/llm/providers/openrouter.go
15. internal/llm/providers/groq.go
16. internal/llm/providers/azure.go
17. internal/llm/providers/copilot.go
18. internal/llm/providers/local.go
19. internal/tools/lsp_client.go
20. internal/tools/lsp_server_manager.go
21. internal/tools/lsp_diagnostics.go
22. internal/tools/lsp_protocol.go
23. internal/tools/diagnostics_tool.go
24. internal/plugins/manager.go
25. internal/plugins/hook.go
26. internal/plugins/loader.go
27. internal/plugins/npm.go
28. internal/plugins/plugin.go
29. internal/plugins/builtins.go
30. internal/plugins/config.go
31. internal/config/local_config.go
32. internal/privacy/telemetry.go
33. internal/indexing/indexer.go
34. internal/indexing/vector_store.go
35. internal/indexing/embedder.go
36. internal/indexing/file_watcher.go
37. internal/indexing/search_tool.go
38. internal/tools/attach_tool.go
39. internal/tools/web_search_tool.go
40. internal/tools/web_fetch_tool.go
41. internal/search/result.go
42. internal/search/summarizer.go
43. internal/commands/custom_command.go
44. internal/commands/registry.go
45. internal/commands/batch.go
46. internal/commands/template.go
47. internal/context/token_counter.go
48. internal/context/truncator.go
49. internal/context/priority.go
50. internal/context/manager.go
51. internal/perf/batcher.go
52. internal/perf/cache.go
53. internal/perf/parallel.go

Modified Files (7 files)

1. cmd/cli/main.go — Add flags for model-switch, custom commands, plugins

2. `internal/llm/llm.go` — Extend Provider interface with HealthCheck, SupportsModel, EstimateCost
3. `internal/config/config.go` — Add LSP, Plugins, Local, Indexing, Commands config
4. `internal/session/session.go` — Integrate PluginManager, HookRegistry, ContextManager, LSPServerManager
5. `internal/tools/registry.go` — Register diagnostics, search, attach, web_search, web_fetch tools
6. `internal/llm/prompt_builder.go` — Include LSP diagnostics and search results in prompts
7. `go.mod` — No new dependencies needed (all already present or stdlib)

Feature Count Summary

#	Feature	New Files	Modified Files	Tests
1	75+ Provider Support with Mid-Session Switching	10	3	5
2	LSP Integration with Diagnostics Feedback	6	2	5
3	Plugin System with 25+ Lifecycle Hooks	7	2	6
4	Local-First Architecture	3	1	5
5	Codebase Indexing	5	1	5
6	Multi-Modal Support	4	1	5
7	Web Search Integration	4	1	5
8	Custom Commands	4	1	5
9	Context Management	4	2	5
10	Performance Optimization	3	1	5
Total	**10 Features**	**50 New**	**15 Modified**	**51 Tests**

End of Complete Porting Plan